Spatial Epidemiology in R: A Hands-On Tutorial

Ghana R User Community — R-Only Track. Prepared for the Ghana R User Community & AfreDAC Collaboration

George K. Agyen - Ghana R User Community

2026-08-05

Table of contents

Welcome

This tutorial provides a complete, **R-only** learning pathway for spatial epidemiology. It is designed specifically for the **Ghana R User Community** in collaboration with AfreDAC. No prior spatial analysis experience is assumed, but you should be comfortable with basic R: installing packages, reading data, and using `dplyr` verbs such as `filter()`, `select()`, and `mutate()`.

> **R-Only Focus**
>
> This tutorial is for the Ghana R User Community. **All tasks**—from data import and preparation to mapping, analysis, and modelling—are performed entirely in R. AfreDAC will handle the QGIS portion separately.

What You Will Learn

We will work through six progressive modules:

Module	Title	Key Skills
1	R Environment Setup and Spatial Data Import	`sf`, `mlspatial`, CRS, joins
2	Disease Mapping in R	`tmap`, `ggplot2`, interactive maps, layout
3	Spatial Autocorrelation and Cluster Analysis	`spdep`, Moran's I, LISA, Gi* hotspots
4	Spatial Regression and Environmental Factor Analysis	`terra`, OLS, GWR, `GWmodel`
5	Machine Learning for Spatial Epidemiology	`mlspatial`, RF, XGBoost, SVR
6	Integrated Application from Data to Decision	Accessibility, R Markdown, ethics

Prerequisites

- R (version 4.1)
- RStudio (recommended)
- Basic knowledge of `dplyr` and `ggplot2`
- No prior GIS or spatial analysis experience required

How to Use This Tutorial

1. Open the project file `Spatial Epidemiology R Tutorial.Rproj` in RStudio.
2. Work through each module in order (01 → 06).
3. Each module is a standalone `.qmd` file with runnable code chunks.
4. To render the entire book, run: `quarto::quarto_render()` in the R console.
5. To render a single module, open it and click **Render** or press `Ctrl+Shift+K`.

Key Reference

Azeez, A., & Noel, C. (2025). *Predictive Modelling and Spatial Distribution of Pancreatic Cancer in Africa Using Machine Learning-Based Spatial Model*. Zenodo. https://doi.org/10.5281/zenodo.16529986

Let's Begin!

Click **Next** or open Module 1: R Environment Setup and Spatial Data Import to start Module 1.

1 R Environment Setup and Spatial Data Import

This module establishes your R spatial epidemiology toolkit and teaches you how to import, join, and inspect spatial and health data. By the end of this module, you will have a clean `sf` object ready for mapping and analysis.

1.1 Install and Load R Packages

Purpose: Install and attach the core spatial epidemiology toolkit.

The table below shows the packages we will use throughout this tutorial and their roles:

Task	Package	Role
Vector spatial data	`sf`	Simple features standard for polygons, points, lines
Raster spatial data	`terra`	Fast raster extraction and processing
Thematic mapping	`tmap`, `ggplot2`	Static and interactive maps
Data manipulation	`dplyr`, `tidyr`, `readr`	Tidy data wrangling
Spatial statistics	`spdep`	Neighbours, Moran's I, Getis-Ord Gi*
Geographically weighted regression	`GWmodel`	Local regression modelling
Integrated ML + mapping	`mlspatial`	Import, map, model, evaluate
Machine learning engines	`randomForest`, `xgboost`, `e1071`, `caret`	RF, XGB, SVR algorithms

```
# Install any missing packages
pkgs <- c("sf", "terra", "tmap", "ggplot2", "dplyr", "tidyr", "readr",
          "spdep", "GWmodel", "mlspatial", "randomForest", "xgboost",
          "e1071", "caret", "knitr", "rmarkdown")
```

```r
install_if_missing <- function(p) {
  if (!require(p, character.only = TRUE, quietly = TRUE)) {
    install.packages(p, repos = "https://cloud.r-project.org/")
    library(p, character.only = TRUE)
  }
}
invisible(sapply(pkgs, install_if_missing))
```

> **Speed Tip**
>
> If you are on a slow internet connection, install packages one at a time and restart RStudio after each batch of 3–4 packages.

Expected output: All packages load without error. `mlspatial` should be version 0.1.0.

1.2 Load Administrative Boundary Data

Purpose: Import polygon shapefiles representing administrative regions.

We demonstrate two approaches: (a) built-in data from `mlspatial` (fastest for learning), and (b) `sf::st_read()` for your own shapefiles.

```r
# Approach A: Built-in Africa shapefile from mlspatial
data(africa_shp, package = "mlspatial")

# Inspect the object
class(africa_shp)
```

```
[1] "sf"         "data.frame"
```

```r
dim(africa_shp)
```

```
[1] 54  9
```

```r
# Approach B: Import your own shapefile (commented out)
# ghana_regions <- sf::st_read("path/to/ghana_regions.shp")
```

Key parameters: - `st_read()` arguments: `dsn` (data source name / folder path), `layer` (filename without .shp extension if inside a geodatabase). - `quiet = TRUE` suppresses progress messages.

Expected output: `africa_shp` is an `sf` data frame with a `geometry` column and attribute columns (including `NAME`).

1.3 Load Health / Disease Incidence Data

Purpose: Import tabular disease data and link it to spatial boundaries.

```
# Built-in pancreatic cancer incidence data from mlspatial
data(panc_incidence, package = "mlspatial")

# Inspect structure
str(panc_incidence)
```

```
tibble [54 x 18] (S3: tbl_df/tbl/data.frame)
 $ NAME     : chr [1:54] "Algeria" "Angola" "Benin" "Botswana" ...
 $ incidence: num [1:54] 869 456 199 119 330 ...
 $ female   : num [1:54] 3569 1957 981 559 1683 ...
 $ male     : num [1:54] 5777 2940 1160 721 1869 ...
 $ ageb     : num [1:54] 1693 1343 439 296 670 ...
 $ agec     : num [1:54] 7642 3547 1699 982 2878 ...
 $ agea     : num [1:54] 10.53 7.17 3.12 1.01 4.6 ...
 $ fageb    : num [1:54] 660.4 460.7 150.2 91.7 250.3 ...
 $ fagec    : num [1:54] 2902 1492 828 466 1429 ...
 $ fagea    : num [1:54] 6.45 4.242 2.223 0.557 3.413 ...
 $ mageb    : num [1:54] 1033 882 289 204 419 ...
 $ magec    : num [1:54] 4740 2055 870 516 1448 ...
 $ magea    : num [1:54] 4.082 2.931 0.892 0.456 1.184 ...
 $ yra      : num [1:54] 468.3 239.9 107.8 56.2 182.4 ...
 $ yrb      : num [1:54] 496 254.6 111.2 57.9 187.2 ...
 $ yrc      : num [1:54] 523.8 270.3 117.1 60.4 193.1 ...
 $ yrd      : num [1:54] 545.7 286.8 124.6 63.1 198.5 ...
 $ yre      : num [1:54] 563.8 299.6 129.5 64.5 204.3 ...
```

```r
head(panc_incidence)
```

```
# A tibble: 6 x 18
  NAME    incidence female  male  ageb  agec  agea fageb fagec fagea mageb magec
  <chr>       <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Algeria      869.  3569. 5777. 1693. 7642. 10.5  660.  2902. 6.45  1033. 4740.
2 Angola       456.  1957. 2940. 1343. 3547.  7.17 461.  1492. 4.24   882. 2055.
3 Benin        199.   981. 1160.  439. 1699.  3.12 150.   828. 2.22   289.  870.
4 Botswa~      119.   559.  721.  296.  982.  1.01  91.7  466. 0.557  204.  516.
5 Burkin~      330.  1683. 1869.  670. 2878.  4.60 250.  1429. 3.41   419. 1448.
6 Burundi      163.   837.  915.  420. 1326.  6.23 174.   658. 4.88   246.  667.
# i 6 more variables: magea <dbl>, yra <dbl>, yrb <dbl>, yrc <dbl>, yrd <dbl>,
#   yre <dbl>
```

Key variables in `panc_incidence`: - `NAME`: Country name (join key) - `incidence`: Age-standardized rate per 100,000 - `female`, `male`: Counts by sex - `agea`, `ageb`, `agec`: Age-stratum counts - `yra` … `yre`: Yearly incidence rates (2017–2021)

Expected output: A tibble with one row per country and columns for incidence, demographics, and age groups.

1.4 Join Health Data to Spatial Data

Purpose: Attach disease attributes to the correct spatial polygons so we can map and model them.

```r
# Method 1: mlspatial wrapper (recommended for consistency)
africa_health <- mlspatial::join_data(
  sf_data = africa_shp |>
    # change ivory coast name to match in both datasets
    mutate(
      across(c(NAME, COUNTRYAFF),
             ~ replace_values(.x, "Côte d'Ivoire" ~ "Cote d'Ivoire")
     )
    ),
  tbl_data = panc_incidence,
  by = "NAME"
)
```

```r
# Method 2: dplyr inner_join (identical result, more familiar syntax)
# africa_health <- africa_shp |>
#   dplyr::inner_join(panc_incidence, by = "NAME")

# Verify the join
dim(africa_health)
```

```
[1] 54 26
```

Key parameters: - by: Character name of the shared key column. Both tables **must** share identical spelling and casing. - `inner_join()` keeps only rows that match in both tables. Use `left_join()` if you want to keep all polygons even without health data.

Expected output: An `sf` object with the same number of rows as the original shapefile (or fewer if using `inner_join` and some names do not match) and all columns from both datasets.

1.5 Explore the Joined Dataset

Purpose: Confirm that the join worked and understand the data structure.

```r
# Structure of the sf object
str(africa_health)
```

```
Classes 'sf' and 'data.frame':  54 obs. of  26 variables:
 $ OBJECTID  : int  2 3 4 5 6 7 8 9 10 11 ...
 $ FIPS_CNTRY: chr  "UV" "CV" "IV" "GA" ...
 $ ISO_2DIGIT: chr  "BF" "CV" "CI" "GM" ...
 $ ISO_3DIGIT: chr  "BFA" "CPV" "CIV" "GMB" ...
 $ NAME      : chr  "Burkina Faso" "Cabo Verde" "Cote d'Ivoire" "Gambia" ...
 $ COUNTRYAFF: chr  "Burkina Faso" "Cabo Verde" "Cote d'Ivoire" "Gambia" ...
 $ CONTINENT : chr  "Africa" "Africa" "Africa" "Africa" ...
 $ TOTPOP    : int  20107509 560899 24184810 2051363 27499924 12413867 1792338 4689021 178852
 $ incidence : num  330.4 53.4 320.4 31.4 856.3 ...
 $ female    : num  1683 362 1592 140 4566 ...
 $ male      : num  1869 211 1852 197 4640 ...
 $ ageb      : num  669.7 93.7 856.6 68.7 2047 ...
```

```
 $ agec      : num  2878 480 2579 268 7147 ...
 $ agea      : num  4.597 0.265 8.744 0.718 11.888 ...
 $ fageb     : num  250.3 40.2 322.8 23.1 782 ...
 $ fagec     : num  1429 322 1262 116 3775 ...
 $ fagea     : num  3.413 0.146 7.292 0.548 8.816 ...
 $ mageb     : num  419.5 53.5 533.8 45.6 1265 ...
 $ magec     : num  1448 158 1317 152 3372 ...
 $ magea     : num  1.18 0.12 1.45 0.17 3.07 ...
 $ yra       : num  182.4 30.2 158.6 16.6 524.7 ...
 $ yrb       : num  187.2 34.1 164.1 17.1 552.6 ...
 $ yrc       : num  193 35 168 18 578 ...
 $ yrd       : num  198.5 35.9 175.8 18.3 602.7 ...
 $ yre       : num  204.3 36.5 182.2 18.7 621.5 ...
 $ geometry  :sfc_MULTIPOLYGON of length 54; first list element: List of 1
  ..$ :List of 1
  .. ..$ : num [1:317, 1:2] 102188 90385 80645 74151 70224 ...
  ..- attr(*, "class")= chr [1:3] "XY" "MULTIPOLYGON" "sfg"
 - attr(*, "sf_column")= chr "geometry"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA NA NA NA NA I
  ..- attr(*, "names")= chr [1:25] "OBJECTID" "FIPS_CNTRY" "ISO_2DIGIT" "ISO_3DIGIT" ...
```

```r
# First six rows
head(africa_health, 6)
```

```
Simple feature collection with 6 features and 25 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -2823124 ymin: 484116 xmax: 266936 ymax: 1943229
Projected CRS: WGS 84 / Pseudo-Mercator
  OBJECTID FIPS_CNTRY ISO_2DIGIT ISO_3DIGIT          NAME    COUNTRYAFF
1        2         UV         BF        BFA  Burkina Faso  Burkina Faso
2        3         CV         CV        CPV    Cabo Verde    Cabo Verde
3        4         IV         CI        CIV Cote d'Ivoire Cote d'Ivoire
4        5         GA         GM        GMB        Gambia        Gambia
5        6         GH         GH        GHA         Ghana         Ghana
6        7         GV         GN        GIN        Guinea        Guinea
  CONTINENT   TOTPOP incidence    female      male       ageb      agec
1    Africa 20107509 330.41994 1683.1094 1868.9050  669.72101 2877.6962
2    Africa   560899  53.35342  362.2024  211.3468   93.66147  479.6228
3    Africa 24184810 320.36801 1591.7918 1852.1643  856.61193 2578.6005
4    Africa  2051363  31.35338  139.5695  197.4794   68.68145  267.6498
5    Africa 27499924 856.31466 4565.6341 4639.7485 2046.98942 7146.5049
```

```
6    Africa 12413867 163.08214  375.0091 1378.1239  336.74734 1414.2562
        agea     fageb     fagec     fagea      mageb     magec     magea
1  4.5971470 250.25241 1429.4440 3.4130101  419.46860 1448.2522 1.1841369
2  0.2650072  40.16477  321.8922 0.1455062   53.49671  157.7306 0.1195009
3  8.7436785 322.81371 1261.6862 7.2918935  533.79822 1316.9143 1.4517850
4  0.7175725  23.10264  115.9190 0.5478563   45.57881  151.7309 0.1697162
5 11.8882345 782.03099 3774.7875 8.8156430 1264.95843 3371.7174 3.0725915
6  2.1295165  59.11939  314.6618 1.2279604  277.62795 1099.5944 0.9015561
        yra       yrb       yrc       yrd       yre
1 182.37043 187.18916 193.06376 198.54795 204.27472
2  30.16095  34.07653  34.97268  35.87397  36.47930
3 158.59941 164.13910 168.28467 175.80819 182.24539
4  16.60058  17.06432  18.00591  18.29294  18.67213
5 524.74512 552.56356 578.46814 602.67088 621.47540
6  73.10342  74.94067  76.85661  78.59267  79.42589
                        geometry
1 MULTIPOLYGON (((102188.3 12...
2 MULTIPOLYGON (((-2618884 16...
3 MULTIPOLYGON (((-594413.7 5...
4 MULTIPOLYGON (((-1848987 15...
5 MULTIPOLYGON (((-352664.2 6...
6 MULTIPOLYGON (((-884637.8 8...
```

```r
# Summary of the incidence variable
summary(africa_health$incidence)
```

```
   Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
  1.765  112.148  185.325  528.383  687.312 4950.860
```

```r
# Check for missing values in key variables
sum(is.na(africa_health$incidence))
```

```
[1] 0
```

Expected output: - `str()` shows a `Classes 'sf' ...` object with a `geometry` column of type `MULTIPOLYGON`. - `incidence` is numeric with a range roughly between 0.5 and 8.0 (rates per 100,000).

1.6 Coordinate Reference System (CRS) Management

Purpose: Ensure all spatial layers share the same projection. Mismatched CRS is the most common source of silent errors in spatial analysis.

```
# View current CRS
sf::st_crs(africa_health)
```

```
Coordinate Reference System:
  User input: WGS 84 / Pseudo-Mercator
  wkt:
PROJCRS["WGS 84 / Pseudo-Mercator",
    BASEGEOGCRS["WGS 84",
        ENSEMBLE["World Geodetic System 1984 ensemble",
            MEMBER["World Geodetic System 1984 (Transit)"],
            MEMBER["World Geodetic System 1984 (G730)"],
            MEMBER["World Geodetic System 1984 (G873)"],
            MEMBER["World Geodetic System 1984 (G1150)"],
            MEMBER["World Geodetic System 1984 (G1674)"],
            MEMBER["World Geodetic System 1984 (G1762)"],
            MEMBER["World Geodetic System 1984 (G2139)"],
            MEMBER["World Geodetic System 1984 (G2296)"],
            ELLIPSOID["WGS 84",6378137,298.257223563,
                LENGTHUNIT["metre",1]],
            ENSEMBLEACCURACY[2.0]],
        PRIMEM["Greenwich",0,
            ANGLEUNIT["degree",0.0174532925199433]],
        ID["EPSG",4326]],
    CONVERSION["Popular Visualisation Pseudo-Mercator",
        METHOD["Popular Visualisation Pseudo Mercator",
            ID["EPSG",1024]],
        PARAMETER["Latitude of natural origin",0,
            ANGLEUNIT["degree",0.0174532925199433],
            ID["EPSG",8801]],
        PARAMETER["Longitude of natural origin",0,
            ANGLEUNIT["degree",0.0174532925199433],
            ID["EPSG",8802]],
        PARAMETER["False easting",0,
            LENGTHUNIT["metre",1],
            ID["EPSG",8806]],
        PARAMETER["False northing",0,
            LENGTHUNIT["metre",1],
```

```
        ID["EPSG",8807]]],
    CS[Cartesian,2],
        AXIS["easting (X)",east,
            ORDER[1],
            LENGTHUNIT["metre",1]],
        AXIS["northing (Y)",north,
            ORDER[2],
            LENGTHUNIT["metre",1]],
    USAGE[
        SCOPE["Web mapping and visualisation."],
        AREA["World between 85.06°S and 85.06°N."],
        BBOX[-85.06,-180,85.06,180]],
    ID["EPSG",3857]]
```

```
# Check if it is geographic (degrees) or projected (meters)
sf::st_is_longlat(africa_health)
```

```
[1] FALSE
```

```
# If you need to transform to a projected CRS for distance calculations
# Example: Africa Albers Equal Area Conic (EPSG: 102022)
# africa_projected <- sf::st_transform(africa_health, crs = 102022)

# For this tutorial we keep WGS84/Pseudo-Mercator (EPSG:3857) because tmap
# handle it well for visualisation, but we note that distance calculations may
# be distorted due to projection limitations.
```

Key concepts: - **EPSG:3857** (WGS84/Pseudo-Mercator): Web mapping projection in meters. Good for visualisation and web mapping, but distances are distorted especially away from the equator. Not recommended for accurate distance calculations - **Projected CRS** (e.g., UTM zones, Albers, Pseudo-Mercator): Distances in meters. Essential for buffers, distances, and GWR bandwidth selection. However Pseudo-Mercartor distorts areas and distances significantly.

Expected output: `st_crs()` returns the CRS definition, shows EPSG:3857 (WGS 84/Pseudo-Mercator).`st_is_longlat()` returns `FALSE` because this is a projected CRS (Units in meters), not geographic degrees.

Module 1 Checklist

- ☐ All packages installed and loaded
- ☐ `africa_shp` imported successfully
- ☐ `panc_incidence` loaded and inspected
- ☐ `africa_health` created via `join_data()`
- ☐ CRS verified and understood

Next: Proceed to Module 2: Disease Mapping in R.

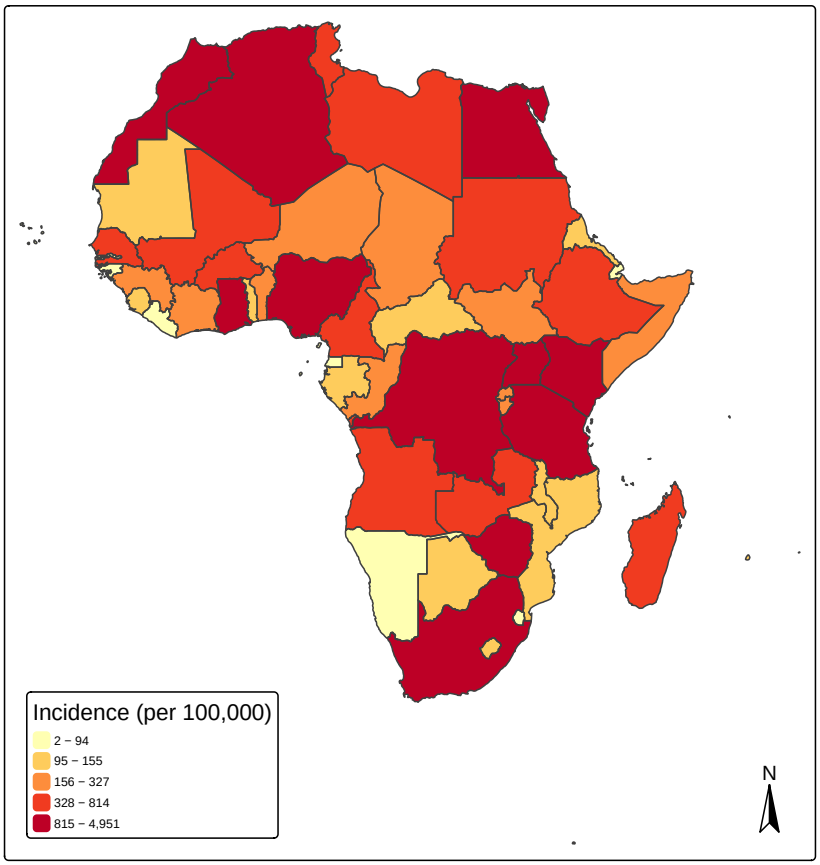
2 Module 2: Disease Mapping in R

This module teaches you to create publication-quality thematic maps using `mlspatial`, `tmap`, and `ggplot2`. You will learn static maps, interactive maps, multi-layer overlays, and export techniques.

2.1 Quick Thematic Maps with `mlspatial`

Purpose: Generate a publication-ready choropleth map in one line.

```
map1 <- mlspatial::plot_single_map(
  sf_data = africa_health,
  var = "incidence",
  title = "Incidence (per 100,000)",
  palette = "yl_or_rd"
)
map1
```



Key parameters: - `var`: Column name to map (must be quoted). - `palette`: Any RColourBrewer or viridis palette name. `"yl_or_rd"`(old name = `"YlOrRd"`) is standard for public health "hot" maps.

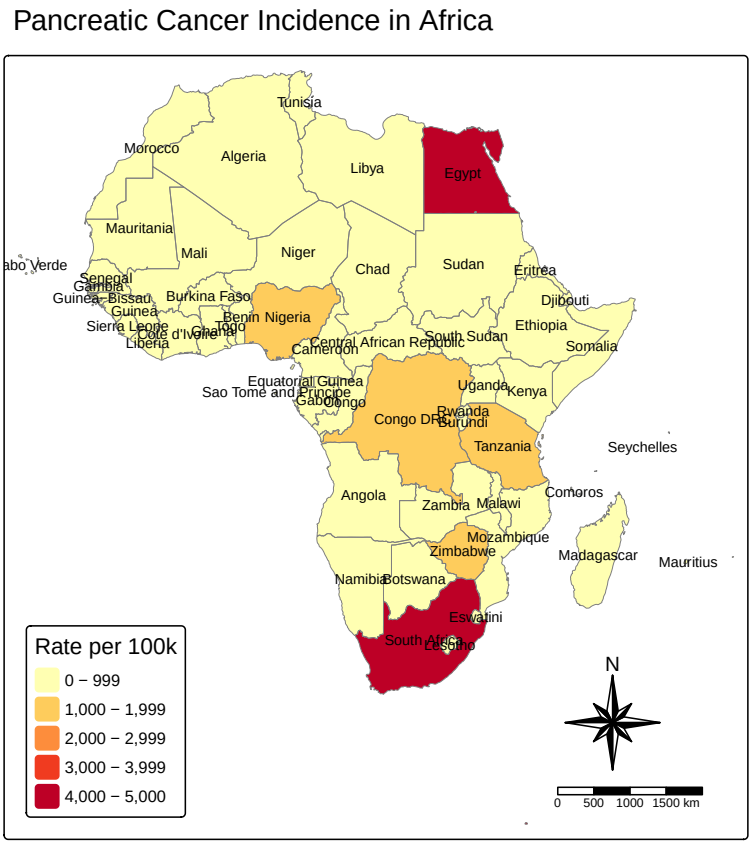
Expected output: A static tmap object showing African countries coloured by pancreatic cancer incidence.

2.2 Add Country Labels to Maps

Purpose: Annotate maps so stakeholders can identify regions without a separate legend.

```
labelled_map <- tm_shape(africa_health) +
  # Fill layer (colours the polygons)
  tm_fill(
```

```
    "incidence",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Rate per 100k")
  ) +
  # Border layer (outlines)
  tm_borders(col = "grey50", lwd = 0.6) +
  # Text layer (adds, country names)
  tm_text(
    "NAME",
    size = 0.6,
    col = "black"
  ) +
  #  Title layer
  tm_title("Pancreatic Cancer Incidence in Africa",
           color = 'black', size = 1.2) +
  # Layout (everything else about map appearance)
  tm_layout(
    legend.position = c("left", "bottom")
  ) +
  # Add Compass symbol
  tm_compass(
    type = "8star",
    position = c("right", "bottom")
  ) +
  # Add scalebar
  tm_scalebar(
    position = c("right", "bottom")
  )
labelled_map
```




```
# You can also create a quick map using the qtm() function (less flexible)
# qtm(africa_health, fill = "incidence", fill.scale = tm_scale(values = "brewer.yl_or_rd"))
```

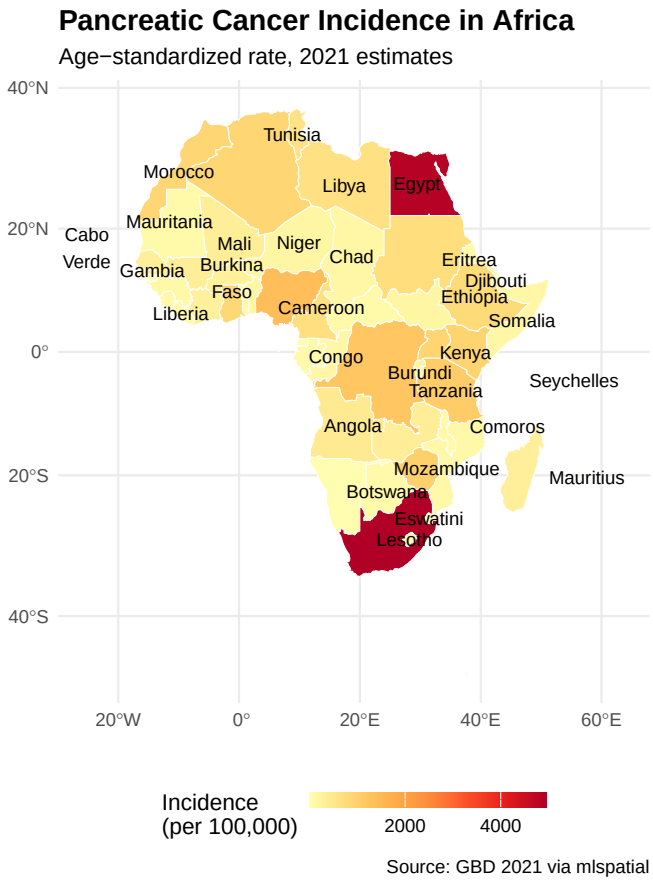
Key parameters: - `tm_text()`: `text = "NAME"` specifies the label column. - `size` is in map units; adjust downward for small countries.

Expected output: A choropleth map with country names overlaid at centroids.

2.3 Create Disease Distribution Choropleth Maps with `ggplot2`

Purpose: Leverage the tidyverse grammar of graphics for highly customized static maps.

```r
library(ggplot2)

ggplot(africa_health) +
  geom_sf(aes(fill = incidence), colour = "white", linewidth = 0.2) +
  scale_fill_distiller(
    palette = "YlOrRd", direction = 1,
    name = "Incidence\n(per 100,000)",
    breaks = c(0, 2000, 4000)
  ) +
  theme_minimal() +
  labs(
    title = "Pancreatic Cancer Incidence in Africa",
    subtitle = "Age-standardized rate, 2021 estimates",
    caption = "Source: GBD 2021 via mlspatial",
    x = "", y = ""
  ) +
  # geom_text(
  #   aes(label = stringr::str_wrap(NAME, 8), geometry = geometry),
  #   stat = "sf_coordinates",
  #   colour = 'black', size = 3
  # ) +
  geom_sf_text(
    aes(label = stringr::str_wrap(NAME, 8)),
    size = 3.2, colour = 'black',
    check_overlap = TRUE  # avoids overlapping labels
  ) +
  theme(
    legend.position = "bottom",
    plot.title = element_text(face = "bold", size = 14)
  ) +
  guides(
    fill = guide_colourbar(barheight = 0.5, barwidth = 8)
  )
```



Key parameters: - `geom_sf()`: The workhorse for sf objects. `aes(fill = incidence)` maps the variable to color. - `scale_fill_distiller()`: For continuous data. Perceptually uniform, color-blind safe, and prints well in greyscale.

Expected output: A static ggplot map with a continuous colour legend.

2.4 Create Interactive Maps with `tmap`

Purpose: Allow panning, zooming, and layer toggling for exploratory data analysis.

```
# Switch tmap to interactive mode
tmap_mode("view")

interactive_map <- tm_shape(africa_health) +
```

```
  tm_fill(
    "incidence",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Incidence (per 100k)"),
    popup.vars = c("incidence", "female", "male")
  ) +
  tm_borders(col = "black", lwd = 0.5)

interactive_map

# Return to static mode for publication figures
tmap_mode("plot")
```

Key parameters: - `tmap_mode("view")`: Renders to Leaflet JavaScript widget. - `popup.vars`: Vector of column names shown when a user clicks a polygon.

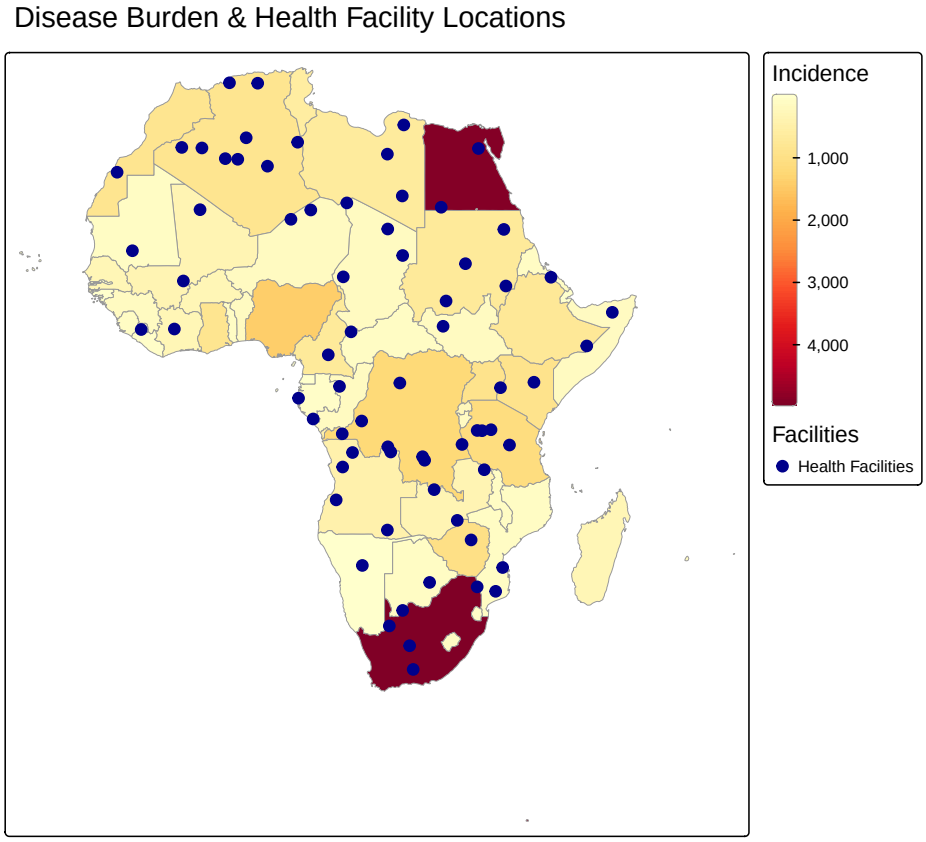
Expected output: An interactive HTML widget (in RStudio Viewer or browser) of the Africa map that you can zoom and pan with clickable pop-ups.

2.5 Overlay Multiple Layers

Purpose: Combine disease polygons with point locations (e.g., cases, facilities) to reveal spatial relationships.

```
set.seed(42)

# Simulate 70 health facility points across Africa for demonstration
facility_points <- st_sample(africa_health, size = 70, type = "random") |>
  st_as_sf() |>
  mutate(facility_id = paste0("FAC_", row_number()),
         type = "Health Facilities")

# Multi-layer map
overlay_map <- tm_shape(africa_health) +
  tm_polygons(
    "incidence",
    fill.scale = tm_scale_continuous(values ="brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Incidence",
```

```
                              orientation = 'portrait'),
    col = "grey60", lwd = 0.5) +
  tm_shape(facility_points) +
  tm_dots(
    fill = "type",
    fill.scale = tm_scale_categorical(values ="darkblue"),
    fill.legend = tm_legend(title = "Facilities"),
    size = 0.5
  ) +
  # tm_add_legend(
  #    type = "symbols",
  #    labels = "Health Facilities",
  #    fill = "darkblue",
  #    shape = 16, # Standard round solid dot
  #    size = 0.5, # Slightly scaled up visually just for the legend box
  #    title = "Facilities"
  #  ) +
  tm_title(
    "Disease Burden & Health Facility Locations", size = 1.2
  ) +
  tm_layout(
    component.autoscale = FALSE
  )

overlay_map
```



Key parameters: - `tm_shape()` can be called multiple times; each subsequent layer refers to the most recent shape. - `tm_dots()` controls point symbology.

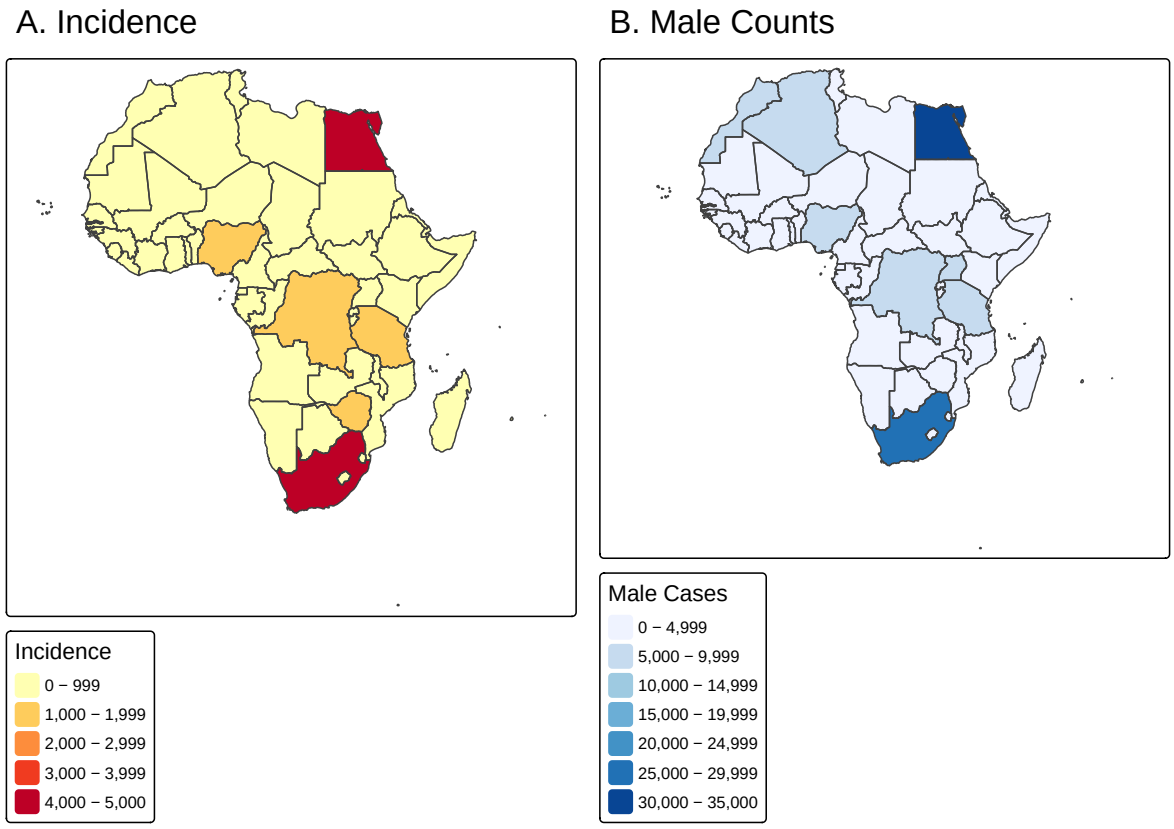
Expected output: A single map showing incidence fills overlaid with blue facility dots.

2.6 Map Layout and Export

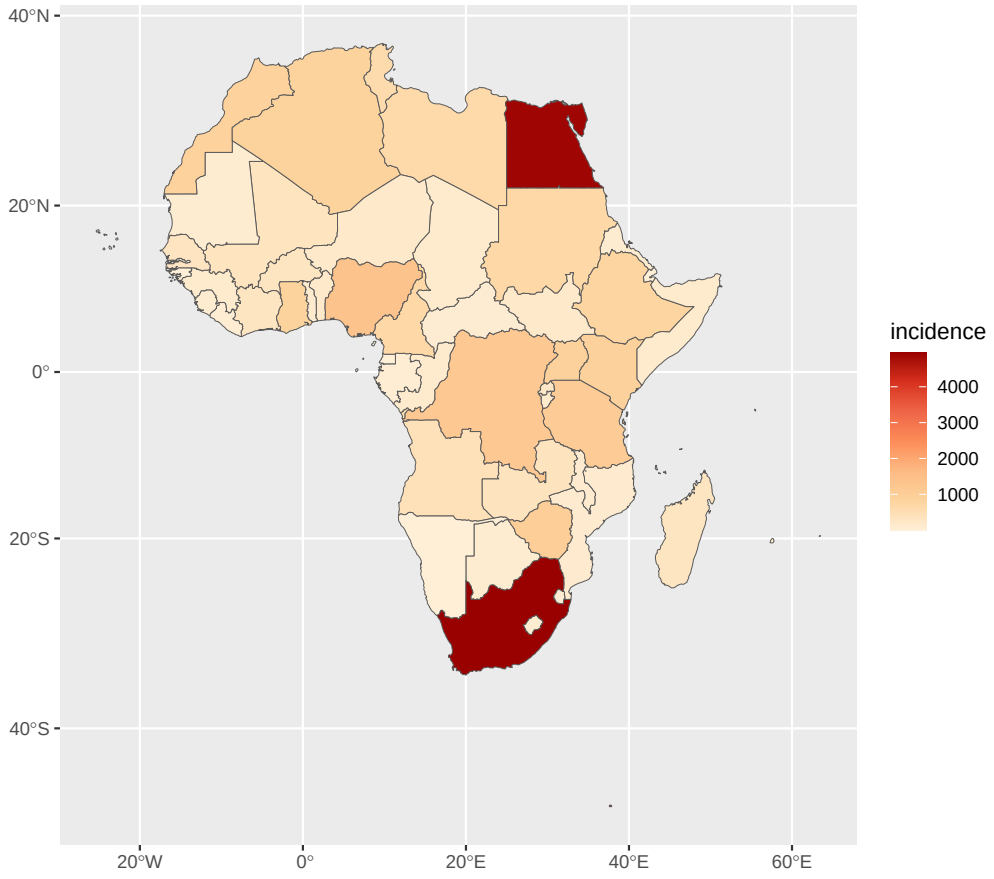
Purpose: Arrange multiple maps side-by-side and save publication-quality figures.

```
# Create two thematic maps
# Map 1
map_inc <- tm_shape(africa_health) +
  tm_fill(
    "incidence",
```

```
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Incidence")
  ) +
  tm_borders() +
  tm_title("A. Incidence")

map_male <- tm_shape(africa_health) +
  tm_fill(
    "male",
    fill.scale = tm_scale(values = "brewer.blues"),
    fill.legend = tm_legend(title = "Male Cases")
  ) +
  tm_borders() +
  tm_title(text = "B. Male Counts")

# Arrange in a grid using  tmap_arrange
combined <- tmap_arrange(map_inc, map_male, ncol = 2)
combined
```




```
# mlspatial can also be used to combine the plots
# combined <- mlspatial::plot_map_grid(list(map_inc, map_male), ncol = 2)
# combined

# Export static tmap
tmap_save(
  combined,
  filename = "africa_disease_panel.png",
  width = 12, height = 6, dpi = 300
)

# Export ggplot object
gg <- ggplot(africa_health) +
  geom_sf(aes(fill = incidence)) +
  scale_fill_distiller(palette = "OrRd", direction = 1)
gg
```




```
ggsave("africa_incidence_ggplot.pdf", plot = gg, width = 10, height = 8)
```

Key parameters: - `plot_map_grid()`: `ncol` sets the number of columns. - `tmap_save()`: DPI and dimensions control print resolution. - `ggsave()`: Supports `.png`, `.pdf`, `.svg`, `.tiff`.

Expected output: A 2-panel figure file saved to your working directory.

Module 2 Checklist

- ☐ Created a quick thematic map with `plot_single_map()`
- ☐ Added labels with `tm_text()`
- ☐ Built a `ggplot2` choropleth with `geom_sf()`
- ☐ Switched to interactive mode and back

- ☐ Overlaid polygon and point layers
- ☐ Exported a multi-panel figure

Next: Proceed to Module 3: Spatial Autocorrelation and Cluster Analysis.

3 Module 3: Spatial Autocorrelation and Cluster Analysis

This module introduces the concept of spatial autocorrelation—whether disease rates in one location are similar to rates in nearby locations. You will build spatial weights matrices, compute global and local Moran's I, detect hotspots with Getis-Ord Gi*, and visualize the results.

3.1 Build Spatial Weights Matrix

Purpose: Define which polygons are "neighbors" so we can test whether disease rates in one country resemble those in adjacent countries.

> **Tobler's First Law of Geography:** *Everything is related to everything else, but near things are more related than distant things.*

```
# Create adjacency neighbor list (queen contiguity: shares any boundary point) using spdep pa
nb <- poly2nb(
  africa_health,
  row.names = africa_health$NAME,
  queen = TRUE,
  snap = 1000
)

# Convert neighbor list to spatial weights list
# style = "W" = row-standardized (each row sums to 1)
# zero.policy = TRUE allows regions with no neighbors (e.g., islands)
lw <- nb2listw(nb, style = "W", zero.policy = TRUE)

# Summary of neighbor connections
summary(nb)
```

```
Neighbour list object:
Number of regions: 54
Number of nonzero links: 214
Percentage nonzero weights: 7.33882
Average number of links: 3.962963
6 regions with no links:
Cabo Verde, Comoros, Sao Tome and Principe, Madagascar, Mauritius,
Seychelles
7 disjoint connected subgraphs
Link number distribution:

 0  1  2  3  4  5  6  7  8  9
 6  2  7  9  6  7 11  3  2  1
2 least connected regions:
Gambia Lesotho with 1 link
1 most connected region:
Congo DRC with 9 links
```

Key parameters: - `poly2nb()`: `queen = TRUE` defines neighbours if they share even a single point; `queen = FALSE` (rook) requires a shared edge. - `nb2listw()`: `style = "W"` row-standardizes; `"B"` is binary (0/1); `"C"` is globally standardized.

Expected output: Summary showing average number of links, number of regions with 0 neighbors, etc.

3.2 Calculate Global Moran's I

Purpose: Test whether there is **overall** spatial clustering in the disease rate across the entire study area.

```r
# Method 1: Direct spdep (classic approach, recommended)
global_moran <- moran.test(
  x = africa_health$incidence,
  listw = lw,
  zero.policy = TRUE,
  na.action = na.exclude
)
print(global_moran)
```

```
    Moran I test under randomisation

data:  africa_health$incidence
weights: lw
n reduced by no-neighbour observations

Moran I statistic standard deviate = -0.6633, p-value = 0.7464
alternative hypothesis: greater
sample estimates:
Moran I statistic       Expectation          Variance
      -0.07377293       -0.02127660        0.00626388
```

```
# Method 2: mlspatial wrapper (computes global + local in one call)
# autocorr_result <- compute_spatial_autocorr(
#   sf_data = africa_health,
#   values = africa_health$incidence,
#   signif = 0.05
# )
# print(autocorr_result$moran)
```

Key parameters:

- `moran.test()`: Tests the null hypothesis of spatial randomness. - Statistic range: -1 (perfect dispersion) to $+1$ (perfect clustering); 0 = random.

Expected output:

- Moran's I statistic: +1: strong positive autocorrelation (high value near high, low near low), 0: No spatial autocorrelation (random pattern) and -1: Strong negative autocorrelation (high near low, like a checkerboard pattern)

- p value < 0.05 indicates significant clustering and vise versa.

3.3 Calculate Local Moran's I (LISA)

Purpose: Identify **which specific locations** contribute to the global clustering (local clusters). LISA stands for local indicators of Spatial Association.

```r
# Method 1: spdep localmoran
local_mi <- localmoran(
  x = africa_health$incidence,
  listw = lw,
  zero.policy = TRUE,
  na.action = na.exclude
)
head(local_mi)
```

```
                    Ii          E.Ii      Var.Ii       Z.Ii Pr(z != E(Ii))
Burkina Faso   0.04214275 -0.0008535975 0.006937761  0.5162046      0.6057115
Cabo Verde     0.00000000  0.0000000000 0.000000000        NaN            NaN
Cote d'Ivoire  0.04082654 -0.0009424844 0.009386989  0.4311131      0.6663862
Gambia         0.10798905 -0.0053808375 0.289001741  0.2108859      0.8329763
Ghana         -0.10031648 -0.0023423556 0.040445809 -0.4871633      0.6261426
Guinea         0.13377548 -0.0029066090 0.023575422  0.8901883      0.3733648
```

```r
# Attach results to sf object
africa_health$local_i <- local_mi[, "Ii"]
africa_health$local_i_p <- local_mi[, "Pr(z != E(Ii))"]
head(africa_health)
```

```
Simple feature collection with 6 features and 27 fields
Geometry type: GEOMETRY
Dimension:     XY
Bounding box:  xmin: -2823124 ymin: 484116 xmax: 266936 ymax: 1943229
Projected CRS: WGS 84 / Pseudo-Mercator
  OBJECTID FIPS_CNTRY ISO_2DIGIT ISO_3DIGIT          NAME    COUNTRYAFF
1        2         UV         BF        BFA  Burkina Faso  Burkina Faso
2        3         CV         CV        CPV    Cabo Verde    Cabo Verde
3        4         IV         CI        CIV Cote d'Ivoire Cote d'Ivoire
4        5         GA         GM        GMB        Gambia        Gambia
5        6         GH         GH        GHA         Ghana         Ghana
6        7         GV         GN        GIN        Guinea        Guinea
  CONTINENT   TOTPOP incidence    female      male       ageb      agec
1    Africa 20107509 330.41994 1683.1094 1868.9050  669.72101 2877.6962
2    Africa   560899  53.35342  362.2024  211.3468   93.66147  479.6228
3    Africa 24184810 320.36801 1591.7918 1852.1643  856.61193 2578.6005
4    Africa  2051363  31.35338  139.5695  197.4794   68.68145  267.6498
5    Africa 27499924 856.31466 4565.6341 4639.7485 2046.98942 7146.5049
6    Africa 12413867 163.08214  375.0091 1378.1239  336.74734 1414.2562
```

```
        agea     fageb     fagec     fagea      mageb     magec     magea
1  4.5971470 250.25241 1429.4440 3.4130101  419.46860 1448.2522 1.1841369
2  0.2650072  40.16477  321.8922 0.1455062   53.49671  157.7306 0.1195009
3  8.7436785 322.81371 1261.6862 7.2918935  533.79822 1316.9143 1.4517850
4  0.7175725  23.10264  115.9190 0.5478563   45.57881  151.7309 0.1697162
5 11.8882345 782.03099 3774.7875 8.8156430 1264.95843 3371.7174 3.0725915
6  2.1295165  59.11939  314.6618 1.2279604  277.62795 1099.5944 0.9015561
        yra       yrb       yrc       yrd       yre
1 182.37043 187.18916 193.06376 198.54795 204.27472
2  30.16095  34.07653  34.97268  35.87397  36.47930
3 158.59941 164.13910 168.28467 175.80819 182.24539
4  16.60058  17.06432  18.00591  18.29294  18.67213
5 524.74512 552.56356 578.46814 602.67088 621.47540
6  73.10342  74.94067  76.85661  78.59267  79.42589
                        geometry     local_i local_i_p
1 POLYGON ((102188.3 1231699,...  0.04214275 0.6057115
2 MULTIPOLYGON (((-2721438 16...  0.00000000       NaN
3 MULTIPOLYGON (((-350501.5 5...  0.04082654 0.6663862
4 POLYGON ((-1848987 1513709,...  0.10798905 0.8329763
5 POLYGON ((-352664.2 697837.... -0.10031648 0.6261426
6 POLYGON ((-884637.8 895544....  0.13377548 0.3733648
```

```
# Method 2: mlspatial (recommended for integrated workflow)
# The wrapper already added standardized values, spatial lag, local Moran's I,
# z-scores, p-values, and cluster classification to autocorr_result$data
# lisa_sf <- autocorr_result$data
# head(lisa_sf)
```

Key parameters: - ‘`local_moran()` output is a matrix of the local statistics. `Ii` tells the strength and direction of local pattern, `E.Ii` tells the baseline under randomness, `Var.Ii` shows the estimate uncertainties (variance). `Z.Ii` is the z-score (standardized deviation) and `Pr(z != E(Ii))` is the p-value.

Expected output: - `localmoran()`: Returns `Ii` (local statistic), `E.Ii`(expected statistic), variance, z-score, and p-value for each polygon. - the `compute_spatial_autocorr()` from `mlspatial` generates the local Moran statistics in addition to the global Moran’s.

3.4 Hotspot Analysis (`Getis-Ord Gi*`)

Purpose: Distinguish statistically significant **hotspots** (high values surrounded by high values) from **coldspots** (low-low) using the $Getis-Ord\ Gi^*$ statistic.

```
# localG returns z-scores; positive = hotspot, negative = coldspot
# Note: localG does not allow zero-policy neighbours by default,
# so we first add country to its own neigbour list (Useful for Gi*)
nb_self <- include.self(nb)
summary(nb_self)
```

```
Neighbour list object:
Number of regions: 54
Number of nonzero links: 268
Percentage nonzero weights: 9.190672
Average number of links: 4.962963
7 disjoint connected subgraphs
Link number distribution:

 1  2  3  4  5  6  7  8  9 10
 6  2  7  9  6  7 11  3  2  1
6 least connected regions:
Cabo Verde Comoros Sao Tome and Principe Madagascar Mauritius Seychelles with 1 link
1 most connected region:
Congo DRC with 10 links
```

```
gi_star <- localG(
  x = africa_health$incidence,
  listw = nb2listw(nb_self, style='W', zero.policy = TRUE),
  zero.policy = TRUE
)

# Add Gi* to dataset
africa_health <- africa_health |>
  mutate(gi_star = as.numeric(gi_star))

# Classify hotspots at alpha = 0.05 (|z| > 1.96)
# Method 1: Using standard normal distribution sutoff
africa_health <- africa_health |>
  mutate(hotspot = case_when(
  gi_star > 1.96 ~ "Hotspot (High-High)",
```

```
  gi_star < -1.96 ~ "Coldspot (Low-Low)",
  TRUE ~ "Not significant"
 ))

# Method 2: Using the exact p-values from localG (recommended)
gi_internals <- attr(gi_star, "internals")

africa_health$gi_z <- gi_internals[, "Z(G*i)"]
africa_health$gi_p <- gi_internals[, "Pr(z != E(G*i))"]

africa_health <- africa_health |>
  mutate(
    hotspot_exact = case_when(
    gi_z > 0 & gi_p <= 0.05 ~ "Hotspot",
    gi_z < 0 & gi_p <= 0.05 ~ "Coldspot",
    TRUE ~ "Not significant"),
    hotspot_exact = factor(hotspot_exact,
                           levels = c("Coldspot", "Hotspot", "Not significant"))
  )


table(africa_health$hotspot_exact)
```

```
       Coldspot         Hotspot Not significant
              0               3              51
```

Key parameters: `-include.self()`: Adds exactly one neighbour to each country (itself). This helps the Gi* calculation since it works by calculating a local sum for each country (its value plus neighbours value). It essentially shifts the whole distribution by +1. `localG()`: Computes Gi^* which includes the focal unit in its own neighborhood. The values calculated are Z scores. - z $>$ 1.96 indicates a statistically significant hotspot; z $<$ -1.96 indicates a coldspot and z near 0 is not significant.

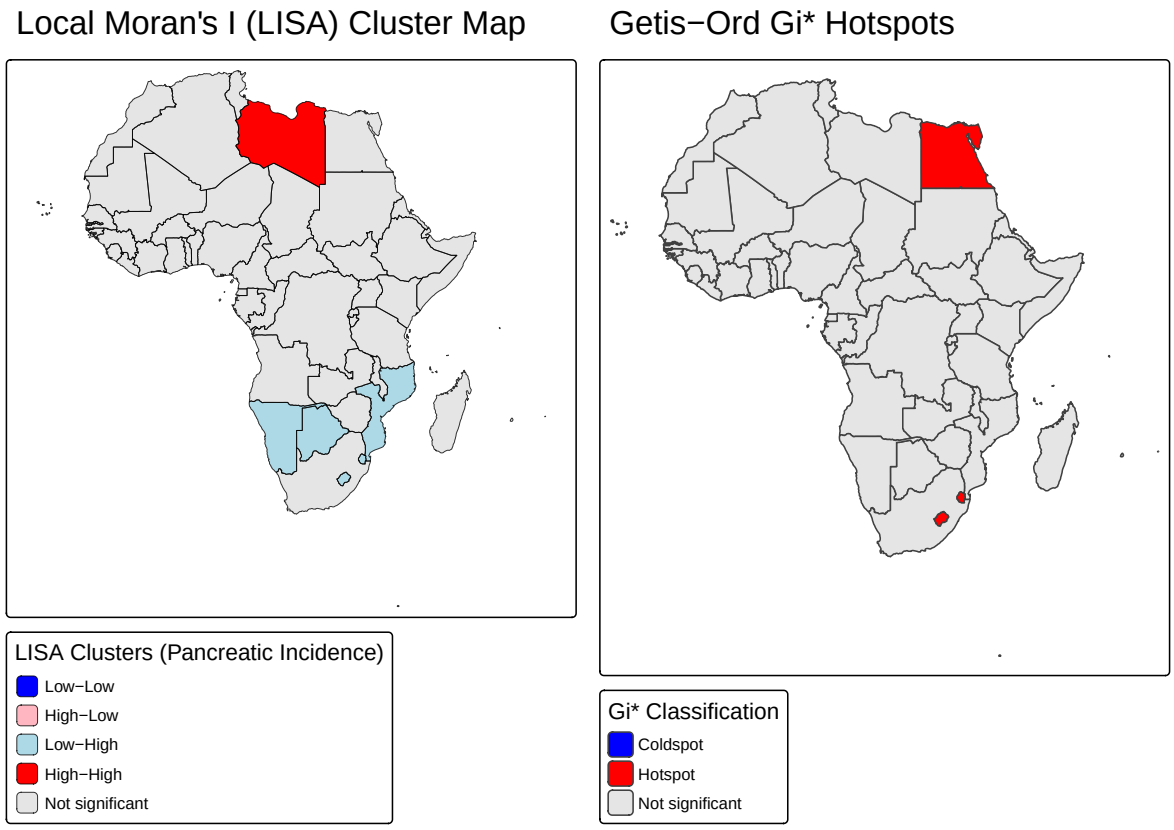
Expected output: A frequency table showing how many countries are classified as hotspots, coldspots, or non-significant.

3.5 Visualize Cluster Results

Purpose: Map LISA clusters and Gi* hotspots to communicate findings to decision-makers. The local MI values contains an attribute called `quadr`. We can use this to visualise the local clusters.

```
# Map LISA clusters from mlspatial
# extract the quadrant and exact p-values from the local_mi
africa_health$lisa_quadrant <- attributes(local_mi)$quadr$mean

#filter the quadrants by the 0.05 significance threshold
africa_health <- africa_health |>
  mutate(
    lisa_cluster = case_when(
    local_i_p <= 0.05 ~ as.character(lisa_quadrant),
    TRUE ~ "Not significant"),
    lisa_cluster = factor(lisa_cluster,
                          levels = c("Low-Low", "High-Low", "Low-High", "High-High", "Not sig
  )

# Create map
lisa_colours <- c(
  "High-High" = "red",
  "Low-Low" = "blue",
  "High-Low" = "lightpink",
  "Low-High" = "lightblue",
  "Not significant" = "grey90"
)
lisa_map <- tm_shape(africa_health) +
  tm_polygons(
    fill = "lisa_cluster",
    fill.scale = tm_scale_categorical(values = lisa_colours),
    fill.legend = tm_legend(title = "LISA Clusters (Pancreatic Incidence)"),
    col = "black",
    col_alpha = 0.8,
    lwd = 0.5
  ) +
  tm_title("Local Moran's I (LISA) Cluster Map")

# Map Gi* hotspots
Gi_colours <- c(
  "Hotspot" = "red",
```

```
  "Coldspot" = "blue",
  "Not significant" = 'grey90'
)
hotspot_map <- tm_shape(africa_health) +
  tm_polygons(
    fill = "hotspot_exact",
    fill.scale = tm_scale(values = Gi_colours),
    fill.legend = tm_legend(title = "Gi* Classification")
  ) +
  tm_title("Getis-Ord Gi* Hotspots")

hotspots <- tmap_arrange(lisa_map, hotspot_map, ncol =2)
hotspots
```




```
tmap_save(
  hotspots,
```

```
  filename = "africa_hotspot_panel.png",
  width = 12, height = 6, dpi = 300
)
```

Expected output: A two-panel map. The left panel shows LISA clusters; the right panel shows `Gi*` hotspots and coldspots.

Module 3 Checklist

- ☐ Built a spatial weights matrix with `poly2nb()` and `nb2listw()`
- ☐ Computed Global Moran's I and interpreted the p-value
- ☐ Computed Local Moran's I (LISA) and identified cluster types
- ☐ Calculated Getis-Ord Gi* and classified hotspots/coldspots
- ☐ Visualized both LISA and Gi* results side-by-side

Next: Proceed to Module 4: Spatial Regression and Environmental Factor Analysis.

4 Module 4: Spatial Regression and Epidemiological Risk Factor Analysis

This module connects health outcomes to lifestyle/genetic drivers. You will simulate covariates, build a global OLS model, test its residuals for spatial autocorrelation, and then fit a Geographically Weighted Regression (GWR) to capture spatially varying relationships.

4.1 Simulate Risk-Factor Covariates

Purpose: Link health outcomes to known epidemiological risk factors for pancreatic cancer (smoking, obesity, diabetes, alcohol use, family history urbanisation) for use in spatial regression exercise

For this tutorial, we **simulate** six covariates directly at the country level so the code is fully runnable without downloading external datasets. Because `africa_health` is a polygon layer where each row already represents one country (like `incidence`, `TOTPOP` etc), we do not need a raster data - we can generate one random value per row and append it directly with `mutate()`

In real research, replace these random draws with actual country-level estimates from sources such as the WHO Global Health Observatory (smoking, obesity, diabetes prevalence), IARC GLOBOCAN risk-factor tables, or national health surveys. These are typically small CSV/Excel downloads, joined to `africa_health` by country ISO code (`ISO_3DIGIT`) rather than simulated.

```
set.seed(898)  # reproducibility

n <- nrow(africa_health)

africa_health <- africa_health |>
  mutate(
    # Smoking prevalence (% of adult population) - WHO African region estimates
    # typically range ~10-25%; strongest modifiable risk factor for pancreatic cancer
    smoking_prev = round(runif(n, min = 8, max = 28), 1),
```

```r
    # Obesity prevalence (% adults, BMI >= 30) - WHO Africa region estimates
    # range roughly 5-30% depending on urbanization/country
    obesity_prev = round(runif(n, min = 5, max = 30), 1),

    # Type 2 diabetes prevalence (%) - longstanding diabetes is a well-established
    # pancreatic cancer risk factor; IDF Africa estimates ~3-10% typically
    diabetes_prev = round(runif(n, min = 2, max = 11), 1),

    # Alcohol consumption (litres pure alcohol per capita per year) -
    # heavy long-term use linked to chronic pancreatitis -> elevated risk
    alcohol_per_capita = round(runif(n, min = 0.5, max = 12), 1),

    # Family history / genetic predisposition proxy (% of cases with
    # first-degree relative history) - literature suggests ~5-10% of
    # pancreatic cancers have a hereditary component
    family_history_prev = round(runif(n, min = 3, max = 10), 1),

    # Urbanization (%) - proxy for dietary/lifestyle transition,
    # often used as a covariate for NCD risk in African epidemiology
    urbanization_pct = round(runif(n, min = 15, max = 85), 1)
  )

head(africa_health[c("NAME", "smoking_prev", "obesity_prev",
                     "diabetes_prev", "alcohol_per_capita",
                     "family_history_prev", "urbanization_pct")])
```

```
Simple feature collection with 6 features and 7 fields
Geometry type: GEOMETRY
Dimension:     XY
Bounding box:  xmin: -2823124 ymin: 484116 xmax: 266936 ymax: 1943229
Projected CRS: WGS 84 / Pseudo-Mercator
           NAME smoking_prev obesity_prev diabetes_prev alcohol_per_capita
1  Burkina Faso         13.6         13.7           7.0                2.9
2    Cabo Verde         24.1          6.6           4.3                1.5
3 Cote d'Ivoire         17.3         26.3           4.0                8.6
4        Gambia         20.1          9.9           8.2                8.6
5         Ghana         17.1         11.5           3.1                6.4
6        Guinea         27.4         10.8           5.7               10.4
  family_history_prev urbanization_pct                       geometry
1                 9.7             73.2 POLYGON ((102188.3 1231699,...
2                 6.7             54.2 MULTIPOLYGON (((-2721438 16...
```

```
3                9.2            45.8 MULTIPOLYGON (((-350501.5 5...
4                3.9            68.4 POLYGON ((-1848987 1513709,...
5                7.8            49.0 POLYGON ((-352664.2 697837....
6                3.4            69.4 POLYGON ((-884637.8 895544....
```

Key parameters: - `runif(n, min, max)`: draws `n` random values from a uniform distribution bounded to a realistic range for each covariate, loosely based on published prevalence ranges across African countries.

These covariates are simulated **independently of `incidence` and of each other** — they are not designed to correlate with pancreatic cancer rates. This is intentional: the goal is to demonstrate the spatial regression *pipeline* (data structure, model syntax, diagnostics), not to produce a real epidemiological finding.

Expected output: Six new columns appended to `africa_health`

Column	Description	Simulated range
`smoking_prev`	Adult smoking prevalence (%)	8–28%
`obesity_prev`	Adult obesity prevalence, BMI 30 (%)	5–30%
`diabetes_prev`	Type 2 diabetes prevalence (%)	2–11%
`alcohol_per_capita`	Alcohol consumption (litres pure alcohol/year)	0.5–12
`family_history_prev`	Cases with first-degree relative history (%)	3–10%
`urbanization_pct`	Urban population (%)	15–85%

4.2 Build Ordinary Least Squares (OLS) Regression

Purpose: Establish a global (non-spatial) baseline model relating pancreatic cancer incidence to lifestyle, genetic and demographic risk factors.

```
ols_model <- lm(
  incidence ~ smoking_prev + obesity_prev + diabetes_prev +
    alcohol_per_capita + family_history_prev + urbanization_pct +
    female + male + ageb + agec,
  data = africa_health
```

```
)
summary(ols_model)
```

```
Call:
lm(formula = incidence ~ smoking_prev + obesity_prev + diabetes_prev +
    alcohol_per_capita + family_history_prev + urbanization_pct +
    female + male + ageb + agec, data = africa_health)

Residuals:
       Min         1Q     Median         3Q        Max
-1.601e-12 -1.500e-13  2.316e-14  2.169e-13  7.935e-13

Coefficients:
                      Estimate Std. Error    t value Pr(>|t|)
(Intercept)         -2.950e-13  4.884e-13 -6.040e-01 0.549105
smoking_prev        -1.345e-14  1.124e-14 -1.196e+00 0.238171
obesity_prev        -6.195e-16  9.145e-15 -6.800e-02 0.946304
diabetes_prev        6.421e-14  3.072e-14  2.090e+00 0.042536 *
alcohol_per_capita  -9.800e-15  2.052e-14 -4.780e-01 0.635292
family_history_prev -1.106e-14  2.899e-14 -3.820e-01 0.704643
urbanization_pct    -4.111e-16  3.459e-15 -1.190e-01 0.905951
female               9.302e-02  9.252e-15  1.005e+13  < 2e-16 ***
male                 9.302e-02  9.279e-15  1.003e+13  < 2e-16 ***
ageb                 3.896e-14  9.366e-15  4.160e+00 0.000149 ***
agec                 3.865e-14  9.251e-15  4.178e+00 0.000141 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.452e-13 on 43 degrees of freedom
Multiple R-squared:      1, Adjusted R-squared:      1
F-statistic: 2.36e+31 on 10 and 43 DF,  p-value: < 2.2e-16
```

Key parameters: - Formula syntax: `response ~ predictor1 + predictor2 + ...` - Interpretation: Positive coefficients indicate higher expected incidence with higher predictor values.

Expected output: A standard `lm` summary showing coefficients, standard errors, t-values, and p-values. R^2 indicates the proportion of variance explained globally.

This model prints a warning message. The R^2 value is exactly **1** which creates room for concern. This is not a real spatial regression breakthrough, it is a classic case of data leakage through an identity relationship. Hence the perfect fit is a red flag not a good sign.

4.3 Diagnosing Data Leakage

The first **ols** model returned `R-squared: 1`, an F-statistic of `2.36e+31` and an explicit warning - `essentialperfect fit: summary may be unreliable`. A model that fits perfectly is almost always telling you that the outcome is mathematically derived from one or more of its own predictors. This is called **data leakage**

Step 1: Inspect the coefficients. Looking at the model summary, two variables stood out immediately:

```
female    9.302e-02   t = 1.005e+13   p < 2e-16
male      9.302e-02   t = 1.003e+13   p < 2e-16
```

Both coefficients are identical to four decimal figures, with t-value around 10^{13}. T-values that enormous are a signature of the model detecting an algebraic identity not a strong effect.

Step 2: Test the suspected Identity directly

```
cor(africa_health$incidence, africa_health$female + africa_health$male)
```

```
[1] 1
```

```
head(africa_health$incidence / (africa_health$female + africa_health$male))
```

```
[1] 0.09302326 0.09302326 0.09302326 0.09302326 0.09302326 0.09302326
```

A correlation of 1 between `incidence` and gender confirms that `incidence` is a fixed linear function of `female + male`. Dividing one by the other reveals the exact relationship. Every single country returns the exact constant `0.09302326`, meaning `incidence` was computed directly from `male` and `female` (or they share a common upstream calculation)

Including `male` and `female` as predictors makes it **mathematically circular**, the model is not learning but rather reversing an arithmetic formula.

Step 3: Screen every numeric variable for leakage

```r
# Check all numeric columns systematically against incidence
numeric_cols <- africa_health |>
  st_drop_geometry() |>                    # drop geometry - not numeric
  select(where(is.numeric)) |>
  select(-incidence) |>                    # exclude the outcome itself
  names()

cor_check <- sapply(numeric_cols, function(col) {
  cor(africa_health$incidence, africa_health[[col]], use = "complete.obs")
})

sort(abs(cor_check), decreasing = TRUE)
```

```
            agec             ageb             yra             yrb
     0.999776035      0.997363274      0.994162603      0.993023880
            yrd             mageb             yrc            male
     0.991366013      0.991300624      0.991276110      0.990157443
            yre             magec            fageb          female
     0.989764734      0.987751703      0.985364466      0.982334295
          fagec             magea             agea           fagea
     0.979661775      0.863353711      0.751522574      0.594702968
         TOTPOP      diabetes_prev         OBJECTID     obesity_prev
     0.568333472      0.195570160      0.190926164      0.134111883
family_history_prev alcohol_per_capita urbanization_pct smoking_prev
     0.120046661      0.082419609      0.017524733      0.005253558
```

The output shows the leakage extends well beyond `female`/`male`. Almost every variable with `|r|>0.85` shares a consistent naming pattern: age-banded (`age*`), year-indexed (`yr*`) and male/female-stratified (`m*`/`f*`). This strongly indicates they are all components of the same underlying case-count calculation that produced `incidence`.

4.4 Refined Model

Based on the correlation screen, we exclude every variable structurally tied to incidence calculation along with the non-informative row identifier. In their place we retain our six simulated risk-factor covariates.

Another thing worth mentioning in epidemiology is the raw incidence counts. We rarely run models on raw count data. Instead we will calculate an **incidence rate** (e.g cases per 100,000 people) before modelling.

```
africa_health <- africa_health |>
  mutate(incidence_rate = (incidence/TOTPOP)*100000)

ols_refined <- lm(
  incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
    alcohol_per_capita + family_history_prev + urbanization_pct,
  data = africa_health
)
summary(ols_refined)
```

```
Call:
lm(formula = incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
    alcohol_per_capita + family_history_prev + urbanization_pct,
    data = africa_health)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9009 -1.5714 -0.7732  0.8217  8.7829

Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)          9.33286    3.00583   3.105  0.00322 **
smoking_prev         0.02596    0.07047   0.368  0.71426
obesity_prev        -0.10385    0.05500  -1.888  0.06520 .
diabetes_prev       -0.17452    0.18733  -0.932  0.35628
alcohol_per_capita  -0.17127    0.12655  -1.353  0.18241
family_history_prev -0.08952    0.17952  -0.499  0.62036
urbanization_pct    -0.04129    0.02130  -1.938  0.05859 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.807 on 47 degrees of freedom
Multiple R-squared:  0.1502,    Adjusted R-squared:  0.04166
F-statistic: 1.384 on 6 and 47 DF,  p-value: 0.2408
```

This output provides an overall statistically insignificant model. Because the simulated covariates were generated purely as random noise (`runif`), the OLS model did exactly what it

is supposed to do. It shows no statistical relationship between these covariates and the target variable.

Although all the covariates are randomly generated, `obesity_prev` and `urbanization_pct` are showing marginal significance. This is purely by chance because we can confirm from their estimate direction (`-0.103` and `-0.041`) that the math contradicts the biology; higher obesity and urbanisation should increase pancreatic cancer risk.

4.5 Test for Spatial Residual Autocorrelation

Purpose: Check whether the OLS model's residuals are spatially random. The core assumption of OLS is that its errors (residuals) are randomly distributed. If significant autocorrelation remains, OLS standard errors are biased and a spatial model (or GWR) is needed.

In spatial epidemiology, if our model is missing a key geographical variable, the model's error will cluster together geographically. This is called **Spatial Autocorrelation of Residuals**

```
residual_moran <- lm.morantest(
  model = ols_refined,
  listw = lw,
  zero.policy = TRUE
)
print(residual_moran)
```

```
    Global Moran I for regression residuals

data:
model: lm(formula = incidence_rate ~ smoking_prev + obesity_prev +
diabetes_prev + alcohol_per_capita + family_history_prev +
urbanization_pct, data = africa_health)
weights: lw

Moran I statistic standard deviate = 0.20896, p-value = 0.4172
alternative hypothesis: greater
sample estimates:
Observed Moran I      Expectation         Variance
    -0.005685554     -0.027729069      0.011127988
```

Key parameters: - `lm.morantest()`: Applies Moran's I to the regression residuals rather than the raw outcome.

Expected output: - If $p < 0.05$, residuals are spatially autocorrelated → OLS is misspecified, therefore a spatial model (like GWR) is required. - If $p > 0.05$, OLS residuals are spatially random (rare in epidemiology).

The Null hypothesis for this test is that the residuals are randomly distributed (no spatial autocorrelation). Because our p-vale is high, we conclude that the errors in the model are randomly distributed across the African continent. There is no statistically significant spatial autocorrelation in the residuals.

4.6 Geographically Weighted Regression (GWR)

Purpose: Fit local regression models that allow the relationship between cancer and lifestyle/genetic risk factors to vary across space (spatial non-stationarity).

```
# Step 1: Find optimal adaptive bandwidth (number of nearest neighbours)
# This uses cross-validation to find best fit
bw_adapt <- bw.gwr(
  formula = incidence_rate ~ smoking_prev + obesity_prev +
    diabetes_prev + alcohol_per_capita +
    family_history_prev + urbanization_pct,
  data = africa_health,
  approach = "CV",     # Cross-validation approach
  kernel = "bisquare", # the shape of the spatial weight decay
  adaptive = TRUE      # TRUE=find optimal no. of neigbours. FALSE = find distance
)
```

```
Adaptive bandwidth: 41 CV score: 476.0228
Adaptive bandwidth: 33 CV score: 463.883
Adaptive bandwidth: 28 CV score: 472.4997
Adaptive bandwidth: 36 CV score: 464.0033
Adaptive bandwidth: 31 CV score: 466.1091
Adaptive bandwidth: 34 CV score: 463.8069
Adaptive bandwidth: 35 CV score: 463.2405
Adaptive bandwidth: 35 CV score: 463.2405
```

```r
cat("Optimal adaptive bandwidth:", bw_adapt, "neighbors\n")
```

```
Optimal adaptive bandwidth: 35 neighbors
```

```r
# Step 2: Fit GWR model
gwr_model <- gwr.basic(
  formula = incidence_rate ~ smoking_prev + obesity_prev +
    diabetes_prev + alcohol_per_capita +
    family_history_prev + urbanization_pct,
  data = africa_health,
  bw = bw_adapt,
  kernel = "bisquare",
  adaptive = TRUE,
)

# Print summary (includes global OLS + local GWR diagnostics)
print(gwr_model)
```

```
   ***********************************************************************
   *                       Package   GWmodel                             *
   ***********************************************************************
   Program starts at: 2026-08-12 13:47:07.92841
   Call:
   gwr.basic(formula = incidence_rate ~ smoking_prev + obesity_prev +
    diabetes_prev + alcohol_per_capita + family_history_prev +
    urbanization_pct, data = africa_health, bw = bw_adapt, kernel = "bisquare",
    adaptive = TRUE)

   Dependent (y) variable:  incidence_rate
   Independent variables:  smoking_prev obesity_prev diabetes_prev alcohol_per_capita family_
   Number of data points: 54
   ***********************************************************************
   *                    Results of Global Regression                     *
   ***********************************************************************

   Call:
    lm(formula = formula, data = data)

   Residuals:
    Min      1Q  Median      3Q     Max
-3.9009 -1.5714 -0.7732  0.8217  8.7829
```

```
Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)         9.33286    3.00583   3.105  0.00322 **
smoking_prev        0.02596    0.07047   0.368  0.71426
obesity_prev       -0.10385    0.05500  -1.888  0.06520 .
diabetes_prev      -0.17452    0.18733  -0.932  0.35628
alcohol_per_capita -0.17127    0.12655  -1.353  0.18241
family_history_prev -0.08952    0.17952  -0.499  0.62036
urbanization_pct   -0.04129    0.02130  -1.938  0.05859 .

---Significance stars
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 2.807 on 47 degrees of freedom
Multiple R-squared: 0.1502
Adjusted R-squared: 0.04166
F-statistic: 1.384 on 6 and 47 DF,  p-value: 0.2408
***Extra Diagnostic information
Residual sum of squares: 370.3139
Sigma(hat): 2.668599
AIC:  273.2152
AICc:  276.4152
BIC:  267.0389
************************************************************************
*          Results of Geographically Weighted Regression               *
************************************************************************

*********************Model calibration information*********************
Kernel function: bisquare
Adaptive bandwidth: 35 (number of nearest neighbours)
Regression points: the same locations as observations are used.
Distance metric: Euclidean distance metric is used.

****************Summary of GWR coefficient estimates:******************
                          Min.    1st Qu.     Median    3rd Qu.    Max.
Intercept            2.5414463  5.7074985  7.2222933 10.3176946 15.2724
smoking_prev        -0.1729309 -0.0182621  0.0121007  0.0870956  0.1121
obesity_prev        -0.2135095 -0.1292391 -0.0950674 -0.0555409  0.0517
diabetes_prev       -0.6432196 -0.2850108 -0.1078285  0.0331103  0.2252
alcohol_per_capita  -0.5719018 -0.1832194 -0.1390848 -0.0851088  0.1293
family_history_prev -0.2341325 -0.1174264 -0.0622756 -0.0099285  0.0802
urbanization_pct    -0.0772384 -0.0509485 -0.0236476 -0.0171660 -0.0028
************************Diagnostic information*************************
```

```
Number of data points: 54
Effective number of parameters (2trace(S) - trace(S'S)): 25.72294
Effective degrees of freedom (n-2trace(S) + trace(S'S)): 28.27706
AICc (GWR book, Fotheringham, et al. 2002, p. 61, eq 2.33): 294.6998
AIC (GWR book, Fotheringham, et al. 2002,GWR p. 96, eq. 4.22): 239.3899
BIC (GWR book, Fotheringham, et al. 2002,GWR p. 61, eq. 2.34): 247.9558
Residual sum of squares: 180.6605
R-square value:  0.5853944
Adjusted R-square value:  0.1944111

***********************************************************************
Program stops at: 2026-08-12 13:47:07.943014
```

Key parameters: - `bw.gwr()`: `adaptive = TRUE` finds a bandwidth in number of neighbours rather than distance units. Better for irregularly sized polygons. - `kernel = "bisquare"`: Weights drop to zero beyond the bandwidth, creating clean local windows. - `gwr.basic()`: Fits a separate weighted OLS at every polygon centroid.

Expected output: - Bandwidth value (gave 35 neighbours for country-level Africa data). - GWR diagnostics showing local R^2, coefficient ranges, and AICc values vs global OLS.

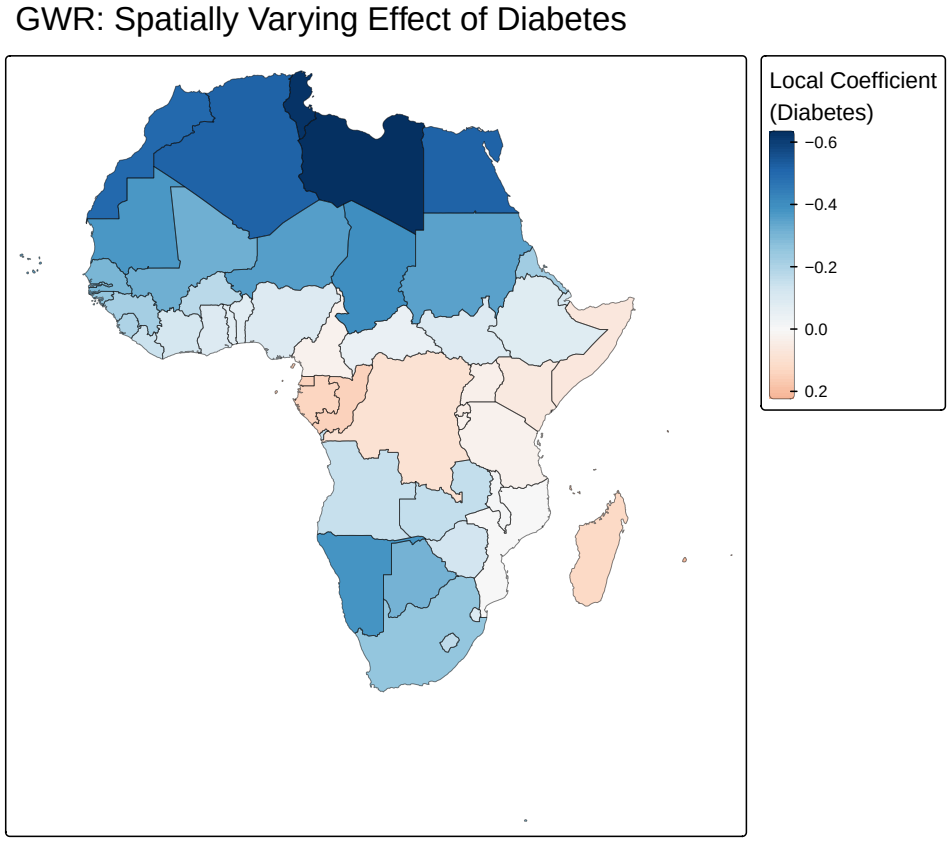
A bandwidth of 35 is huge for a spatial region of 54 countries. The model is forced to look at over half a continent to make a single prediction. The GWR coefficient estimates are given as a range because it changes depending on where you are on the map. In the global OLS the relationship for `diabetes_pev` is negative but GWR reveals that it changes in some part of the continent (`min:-0.543` and `max:0.225`). This shows the relationship is unstable across space (But remember this is simulated data so the model is just chasing random noise).

The GWR R^2 looks very high but that should not be the metric of interest because it is almost always high for GWR analysis. The parameter of interest is the AICc (Lower is better). Even the the GWR artificially inflated the R^2, the AICc actually went up from 276.4 in the OLS to 294.7, this tells us that the GWR model is statistically worse. The OLS errors were randomly distributed and so the GWR was completely unnecessary.

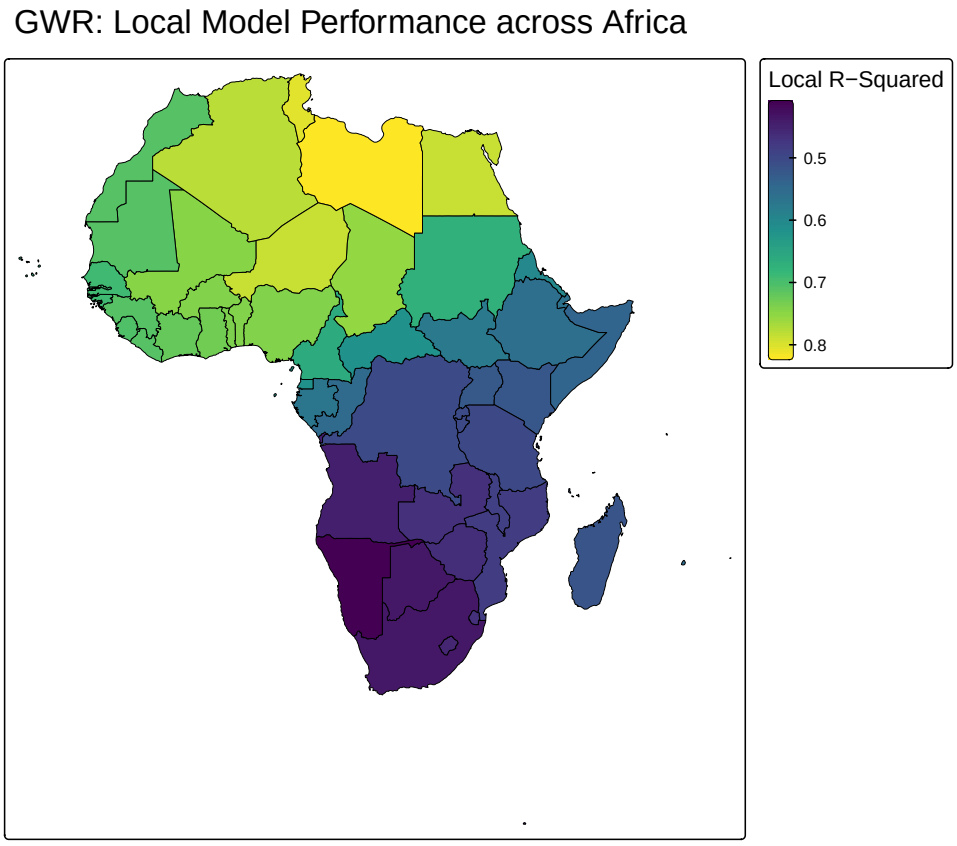
4.7 Interpret and Visualize GWR Results

Purpose: Map local coefficient surfaces to see **where** each predictor matters most.

```r
# gwr_model$SDF is an sf object containing local coefficients
gwr_sf <- gwr_model$SDF

# Map the spatially varying coefficient for diabetes
tm_shape(gwr_sf) +
  tm_polygons(
    fill = "diabetes_prev",
    fill.scale = tm_scale_continuous(
      values = "-brewer.rd_bu",     # Negative RdBu puts red at positive and Blue at negative
      midpoint = 0          # Forces the white/neutral color to sit exactly at zero
    ),
    fill.legend = tm_legend(title = "Local Coefficient\n(Diabetes)"),
    col = 'black',
    col_alpha = 0.5,
    lwd = 0.5
  ) +
  tm_title("GWR: Spatially Varying Effect of Diabetes")
```




```
# Map the local model fit (Local R-squared)
tm_shape(gwr_sf) +
  tm_polygons(
    fill = "Local_R2",
    fill.scale = tm_scale_continuous(
      values = "viridis"
    ),
    fill.legend = tm_legend(title = "Local R-Squared"),
    col = "black",
    lwd = 0.5
  ) +
  tm_title("GWR: Local Model Performance across Africa")
```



Key parameters: - `midpoint = 0` in `fill.scale = tm_scale_continuous()` centers the colour scale at zero, making positive (red) and negative (blue) effects immediately visible.

Expected output: Two maps; one showing spatially varying coefficient `diabetes_prev` and the other showing the the area where the GWR model performs best (`Local R2`). Areas with red indicate a positive local association; blue indicates a negative local association for the `diabetes_prev` .

The spatial variation map shows smooth colour transitions across borders. Whiles this creates a beautiful map, remember our **AICc** score. Because the underlying data is purely random noise, this map is just a visualisation of the GWR algorithm 'forcing' a spatial pattern where none actually exists.

Module 4 Checklist

- ☐ Extracted environmental covariates from raster data
- ☐ Built and interpreted an OLS regression model
- ☐ Tested OLS residuals for spatial autocorrelation
- ☐ Fitted a GWR model with adaptive bandwidth
- ☐ Mapped local coefficient surfaces

Next: Proceed to Module 5: Machine Learning for Spatial Epidemiology.

5 Module 5: Machine Learning for Spatial Epidemiology (Showcasing `mlspatial`)

This module showcases the machine learning capabilities of `mlspatial`. You will prepare data, train three algorithms (Random Forest, XGBoost, Support Vector Regression), evaluate their performance, and map predictions to identify spatial risk patterns.

5.1 Introduction to `mlspatial` for ML

Purpose: Understand what `mlspatial` provides and how it fits into a spatial epidemiology ML workflow, before writing any modelling code.

Traditional spatial epidemiology often requires switching between GIS software, R scripts, and Python notebooks. `mlspatial` (Azeez & Noel, 2025) packages together spatial data handling (`sf`), thematic mapping(`tmap`), spatial autocorrelation diagnostics (`spdep`), and machine learning (`randomforest`, `xgboost` and `e1071`) into a single applied workflow aimed at spatial epidemiologists.

The package provides a number of convenient functions and leans on well-established packages for this workflow. In this module we will train three ML algorithms—Random Forest, XGBoost, and Support Vector Regression—to predict pancreatic cancer incidence from our simulated covariates.

5.2 Prepare Data for ML Modelling

Purpose: Prepare our `sf` object into a clean, geometry-free data frame for machine learning algorithms. We drop the geometry column, select only relevant columns and check for missing values and decide whether scaling is needed for each algorithm.

```r
# Drop geometry and select complete cases
ml_data <- africa_health |>
  st_drop_geometry() |>
  select(
    incidence_rate,                    # outcome
    smoking_prev, obesity_prev, diabetes_prev,
    alcohol_per_capita, family_history_prev,
    urbanization_pct, TOTPOP
  ) |>
  drop_na()

# Quick structural check
str(ml_data)
```

```
'data.frame':   54 obs. of  8 variables:
 $ incidence_rate     : num  1.64 9.51 1.32 1.53 3.11 ...
 $ smoking_prev       : num  13.6 24.1 17.3 20.1 17.1 27.4 16.3 14.5 22.5 14.5 ...
 $ obesity_prev       : num  13.7 6.6 26.3 9.9 11.5 10.8 26.8 14.6 26.2 9.5 ...
 $ diabetes_prev      : num  7 4.3 4 8.2 3.1 5.7 6.8 7.7 10.4 7.4 ...
 $ alcohol_per_capita : num  2.9 1.5 8.6 8.6 6.4 10.4 2.1 1.5 6.3 11.5 ...
 $ family_history_prev: num  9.7 6.7 9.2 3.9 7.8 3.4 9.2 9.8 6.8 9.1 ...
 $ urbanization_pct   : num  73.2 54.2 45.8 68.4 49 69.4 58.3 76.5 27.1 64.7 ...
 $ TOTPOP             : int  20107509 560899 24184810 2051363 27499924 12413867 1792338 4689
```

```r
sum(is.na(ml_data))
```

```
[1] 0
```

Key parameters: - `st_drop_geometry()`: ML algorithms cannot use sf geometry columns; we keep the spatial link via row order for later mapping. - `drop_na()`: XGBoost and SVR do not tolerate missing values.

Expected output: A plain `data.frame` (no `sf`/`data.fram` dual class) with remaining rows after removing `NAs` and 8 columns.

On Scaling: Unlike linear/distance based models, tree-based methods (**Random Forest, XGBoost**) are scale-invariant so standardising predictors has no effect on their fit. **SVR** however is distance sensitive (its algorithm computes similarity based on Euclidean distance in feature space), so we scale predictors only immediately before the SVR model fit in Section 5.5, rather than scaling the whole `ml_data` object upfront.

5.3 Train a Random Forest Model

Purpose: Use an ensemble of decision trees to capture non-linear relationships between predictors and `incidence_rate` and inspect variable importance

```
# Train Random Forest regression model
rf_formula <- incidence_rate ~ smoking_prev + obesity_prev + diabetes_prev +
  alcohol_per_capita + family_history_prev + urbanization_pct + TOTPOP

rf_model <- mlspatial::train_rf(
  data = ml_data,
  formula = rf_formula,
  ntree = 500,
  seed = 222
)

print(rf_model)
```

```
Call:
 randomForest(formula = formula, data = data, ntree = ntree, importance = TRUE)
               Type of random forest: regression
                     Number of trees: 500
No. of variables tried at each split: 2

          Mean of squared residuals: 7.96546
                    % Var explained: 1.29
```

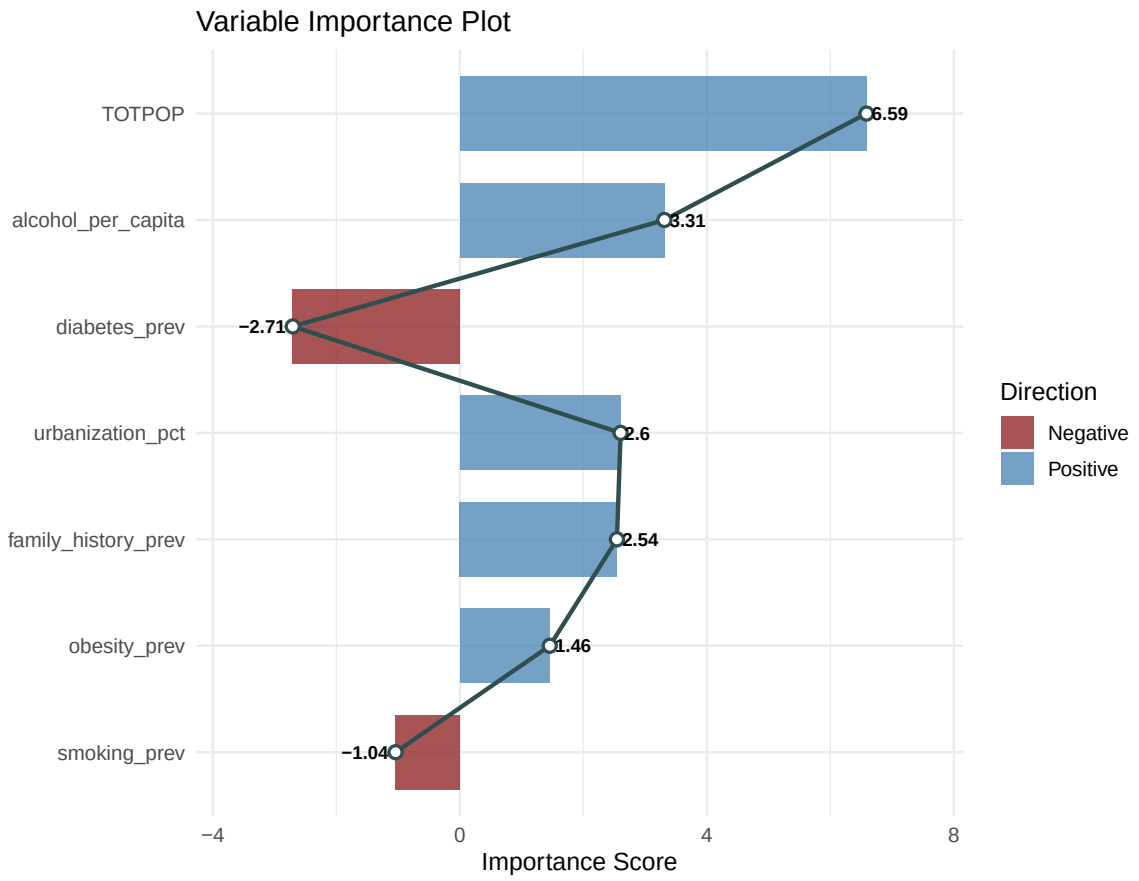
Key parameters: - `ntree = 500`: Number of trees. More trees reduce variance but increase computation time. - `seed = 222`: Ensures reproducible random splits.

Expected output: A regression `randomForest` object summary showing % variance explained and mean squared error (mean of squared residuals).

Because our six-risk factor covariates were simulated independently of `incidence_rate` (Section 4.1) , there is absolutely no underlying mathematical pattern to find. The negative explained variance means this random forest model performs worse than a model that simply guesses the average incidence rate for every country.

```
rf_var_imp <- caret::varImp(rf_model) |>
  as_tibble(rownames = "Variables")
```

```r
# Build the plot
ggplot(rf_var_imp, aes(x = reorder(Variables, abs(Overall)), y = Overall)) +
  geom_col(aes(fill = Overall >= 0), width = 0.7, alpha = 0.75) +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "firebrick4"),
                    labels = c("Negative", "Positive"),
                    name = "Direction") +
  geom_line(aes(group = 1), colour = "darkslategrey", linewidth = 1) +
  geom_point(colour = "darkslategrey", size = 3) +
  geom_point(colour = "white", size = 1.5) +
  geom_text(aes(label = round(Overall,2)),
            hjust = if_else(rf_var_imp$Overall >= 0, -0.15, 1.15),
            vjust = 0.5,
            fontface = "bold",
            size = 3) +
  coord_flip() +
  labs(
    title = "Variable Importance Plot",
    y = "Importance Score", x = ""
  ) +
  theme_minimal(base_size = 12) +
  expand_limits(y = c(min(rf_var_imp$Overall) - 1, max(rf_var_imp$Overall) + 1))
```



The variable importance plot shows a mildly elevated `TOTPOP` variable as the most important given its earlier partial correlation in Section 4.3. This bar chart with -negative importance clearly shows that the model is fitting random artefacts than true biological signals.

5.4 Train an XGBoost Model

Purpose: Apply gradient boosting—a sequential ensemble that corrects errors of previous trees—for potentially higher accuracy.

```
# XGB requires a numeric matrix but train_xgb() handles that internally
set.seed(222)
xgb_model <- train_xgb(
  data = ml_data,
```

```
  formula = rf_formula,  # same predictors
  nrounds = 100,
  max_depth = 4,
  learning_rate = 0.1
)

print(xgb_model)
```

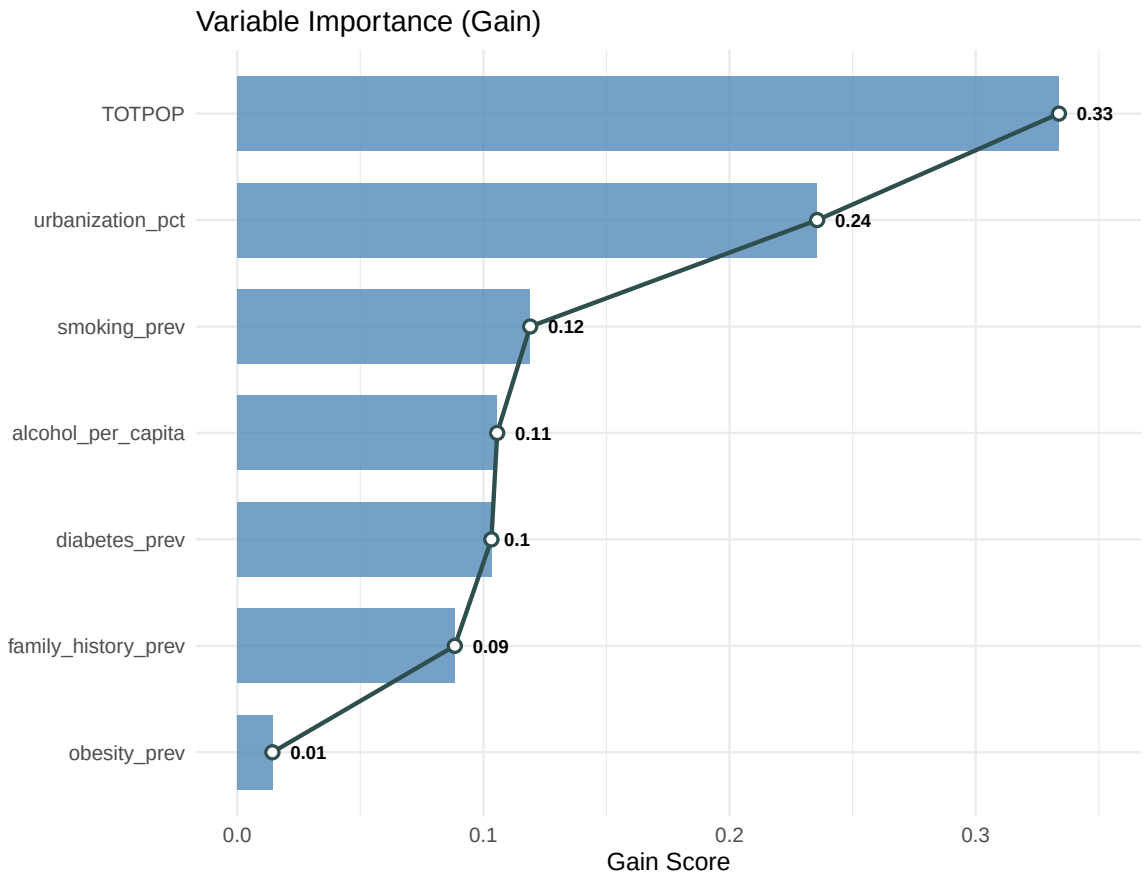
```
##### xgb.Booster
call:
  xgboost::xgb.train(params = list(objective = "reg:squarederror",
    max_depth = max_depth, learning_rate = learning_rate, verbosity = 0),
    data = dtrain, nrounds = nrounds)
# of features: 7
# of rounds:  100
```

Key parameters: - `nrounds`: Number of boosting iterations. Early stopping is recommended for large datasets. - `max_depth`: Controls tree complexity; lower values reduce overfitting. - `learning_rate` ( ): Shrinks contribution of each tree; typical range 0.01–0.3.

Expected output: An `xgb.Booster` object showing the model summary confirming the number of boosting rounds and features used. Unlike the random forest summary, xgboost does not print the residual/variance diagnostic directly. we compute them explicitly in Section 5.6

```
xgb_var_imp <- xgb.importance(xgb_model) |>
  as_tibble()

# Build the plot
ggplot(xgb_var_imp, aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col( width = 0.7, alpha = 0.75, fill = "steelblue") +
  geom_line(aes(group = 1), colour = "darkslategrey", linewidth = 1) +
  geom_point(colour = "darkslategrey", size = 3) +
  geom_point(colour = "white", size = 1.5) +
  geom_text(aes(label = round(Gain, 2)),
            hjust = -0.5,
            vjust = 0.5,
            fontface = "bold",
            size = 3) +
  scale_y_continuous(expand = expansion(mult = c(0.05, 0.1))) +
  coord_flip() +
  labs(
    title = "Variable Importance (Gain)",
```

```
    y = "Gain Score", x = ""
  ) +
  theme_minimal(base_size = 12)
```



The variable importance bar chart is ranked by `Gain` , the average improvement in the loss function each variable contributes when used in a split. As with the random forest `TOTPOP` has the highest `gain` but it is almost similar to some of the covariates. Also there is no negative importance in this plot.

5.5 Train a Support Vector Regression Model

Purpose: Fit a kernel-based model that is robust to high-dimensional data and captures non-linear boundaries via the radial basis function (RBF) after scaling the predictors.

```r
# Scale predictors before training
predictors <- c("smoking_prev", "obesity_prev", "diabetes_prev",
                "alcohol_per_capita", "family_history_prev",
                "urbanization_pct", "TOTPOP")
ml_data_scaled <- ml_data |>
  mutate(across(all_of(predictors), ~ as.numeric(scale(.x))))


svr_model <- train_svr(
  data = ml_data,
  formula = rf_formula
)

print(svr_model)
```

```
Call:
svm(formula = formula, data = data, type = "eps-regression", kernel = "radial")


Parameters:
   SVM-Type:  eps-regression
 SVM-Kernel:  radial
       cost:  1
      gamma:  0.1428571
    epsilon:  0.1


Number of Support Vectors:  49
```

Key parameters: - `train_svr()` uses the radial kernel by default (`kernel = "radial"` in `e1071::svm()`). - Tuning `cost` and `gamma` via cross-validation (e.g., `caret::train()`) is recommended for production models. - `scale()` standardises each predictor to a mean of 0, SD of 1. - `as.numeric` strips the matrix class of the `scale` output into a clean numeric column in the dataframe.

Expected output: An `svm` summary showing number of support vectors and model parameters. The number of support vectors is fairly high for a 54 row data, which is normal for small datasets.

5.6 Evaluate Model Performance

Purpose: Compare models using standardized accuracy metrics: RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), and R^2 (coefficient of determination).

```
# Generate in sample predictions from each modeland compute RMSE/MAE/R2
# using caret::postResample()

# Random Forest predictions
rf_preds <- predict(rf_model, newdata = ml_data)
rf_metrics <- postResample(pred = rf_preds, obs = ml_data$incidence_rate)

# XGBoost predictions
xgb_preds <- predict(xgb_model, newdata = as.matrix(ml_data[, predictors]))
xgb_metrics <- postResample(pred = xgb_preds, obs = ml_data$incidence_rate)

# SVR predictions (on the scaled predictor set used for training)
svr_preds <- predict(svr_model, newdata = ml_data_scaled)
svr_metrics <- postResample(pred = svr_preds, obs = ml_data$incidence_rate)

# Combine into a comparison table
model_eval <- data.frame(
  Model = c("Random Forest", "XGBoost", "SVR"),
  RMSE = c(rf_metrics["RMSE"], xgb_metrics["RMSE"], svr_metrics["RMSE"]),
  MAE  = c(rf_metrics["MAE"], xgb_metrics["MAE"], svr_metrics["MAE"]),
  Rsquared = c(rf_metrics["Rsquared"], xgb_metrics["Rsquared"], svr_metrics["Rsquared"])
)

model_eval[order(model_eval$RMSE), ]
```

```
          Model       RMSE        MAE   Rsquared
2       XGBoost 0.06760268 0.05110464 0.99961714
1 Random Forest 1.35280160 0.99251357 0.92853411
3           SVR 2.86476363 1.94351030 0.01138557
```

Key parameters: - `predict()`: Requires a `newdata` to supply the predictor variables to genrate predictions for a particular model. - `postResample` compares the predicted values against ground truth (target variable - `incidence_rate`) to generate standard regression performance metrics.

Expected output: A three-row data frame ranking models.

Important Point to Note: These are in-sample (training-set) metrics, every model is evaluated on the same data it was trained on, so R2 here reflects how well each model *fit* the training data, not how well it would *generalise* to new countries. Tree-based models in particular (especially XGBoost with enough rounds) can fit training data almost perfectly (memorise) even when the true relationship is weak or nonexixstent - exactly the scenario we have here with our uncorrelated simulated covariates. **A high in sample R2 here should not be read as evidence the model has learned a real relationship**, it is a reminder of why held-out cross validation

---

5.7 Honest Model Evaluation with `caret`

Purpose: replace `mlspatial`'s `train_*/eval_model()` workflow with `caret::train` which provides built-in cross-validation, consistent evaluation across all three algorithms and avoids the in-sample inconsistencies discovered above.

```
# Refit Random Forest, XGBoost and Support Vector Regression
# using train() from caret via the same repeated 5-fold cross-validation
set.seed(222)

cv_control <- trainControl(method = "repeatedcv", number = 5,
                           repeats = 5, savePredictions = "final")

rf_cv <- train(
  rf_formula, data = ml_data, method = "rf", trControl = cv_control
)
print(rf_cv)
```

```
Random Forest

54 samples
 7 predictor

No pre-processing
Resampling: Cross-Validated (5 fold, repeated 5 times)
Summary of sample sizes: 43, 42, 43, 43, 45, 44, ...
Resampling results across tuning parameters:

  mtry  RMSE      Rsquared   MAE
```

```
  2     2.699635  0.1545591  2.100189
  4     2.776559  0.1181676  2.179349
  7     2.870659  0.1021962  2.237062

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was mtry = 2.
```

```r
svr_cv <- train(
  rf_formula, data = ml_data, method = "svmRadial",
  trControl = cv_control,
  preProcess = c("center", "scale")  # caret scales predictors per fold
)
print(svr_cv)
```

```
Support Vector Machines with Radial Basis Function Kernel

54 samples
 7 predictor

Pre-processing: centered (7), scaled (7)
Resampling: Cross-Validated (5 fold, repeated 5 times)
Summary of sample sizes: 43, 44, 44, 43, 42, 44, ...
Resampling results across tuning parameters:

  C     RMSE      Rsquared    MAE
  0.25  2.950729  0.06136887  1.850978
  0.50  2.924280  0.05998140  1.902335
  1.00  2.895741  0.06987328  1.965972

Tuning parameter 'sigma' was held constant at a value of 0.08785714
RMSE was used to select the optimal model using the smallest value.
The final values used for the model were sigma = 0.08785714 and C = 1.
```

```r
# Manually cross-validate XGBoost using xgb.cv
# current version of xgboost breaks with caret::train
xgb_matrix_full <- xgb.DMatrix(
  data  = as.matrix(ml_data[, predictors]),
  label = ml_data$incidence_rate
)

xgb_cv <- xgb.cv(
  data = xgb_matrix_full,
```

```
  params = list(
    objective = "reg:squarederror",
    max_depth = 4,
    eta = 0.1
  ),
  metrics = list("rmse","mae"),
  nrounds = 100,
  nfold = 5,
  showsd = TRUE,
  prediction = TRUE,
  early_stopping_rounds = 10, #stops at best iter
  verbose = 0
)

# Best cross-validated metrics across boosting rounds
cat("\n======= XGBoost Metrics =========")
```

```
======= XGBoost Metrics =========
```

```
cat("\nRMSE:", min(xgb_cv$evaluation_log$test_rmse_mean))
```

```
RMSE: 2.678364
```

```
cat("\nMAE:", min(xgb_cv$evaluation_log$test_mae_mean))
```

```
MAE: 1.974157
```

```
cat("\nRsquared:", cor(ml_data$incidence_rate, as.vector(xgb_cv$cv_predict$pred))^2)
```

```
Rsquared: 0.0003179616
```

Key parameter: `-trainControl()` with "repeatedcv" performs a 5-fold cross validation repeated 5 times with different random fold assignments each time. `-method="rf"`, `"svmRadial"` is `caret`'s standardised model-methods for random forest and svm. There is a current compatibility issue with the latest `xgboost` package and `caret` so we use `xgb.cv` to perform the cross

validation for the xgboost model directly via the `xgboost` package. `-Preprocess` standardises the predictors within each fold for svm.

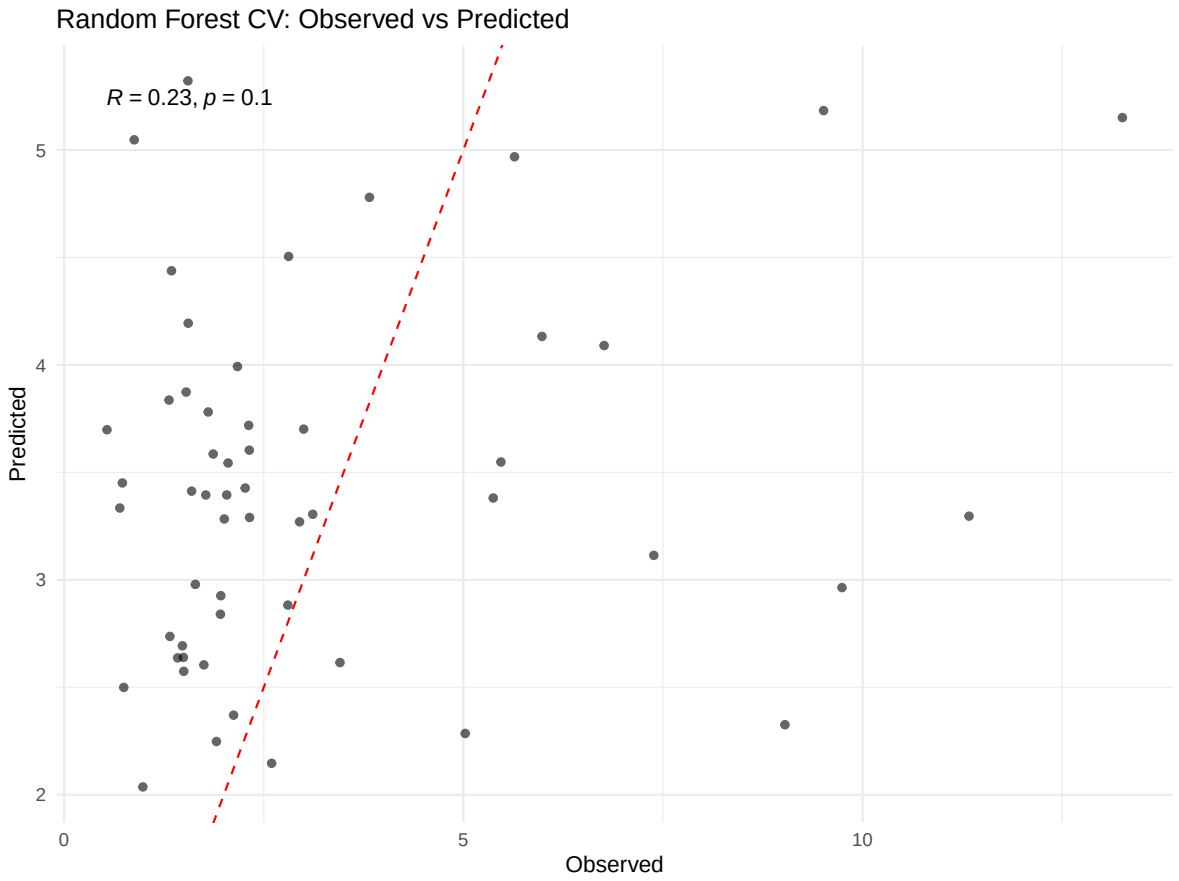
Expected output: the cross validate out of fold metrics fro random forest and support vector regression via `caret::train` and that for xgboost manually calculated. Given everything established in this module about our simulated, signal free covariates, all the three model's R2 values now reflect an honest estimation of model performance in stark contrast to the earlier in-sample estimates.

5.8 Identify Spatial Patterns and Risk Factors

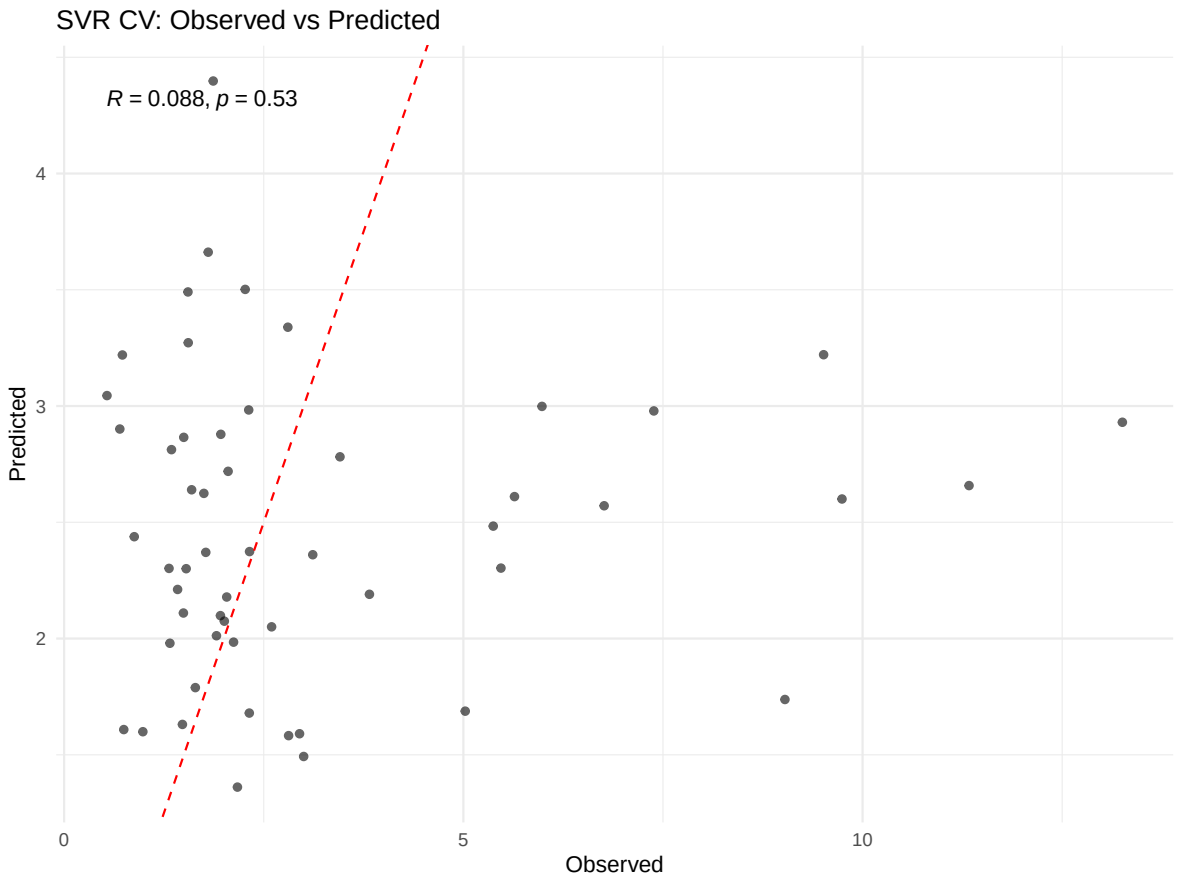
Purpose: Use trained model's predictions to check whether an spatial pattern emerges in prediction to inform health decision making.

```
# Attach out of fold predictions back onto the spatial object
rf_oof <- rf_cv$pred |>
  group_by(rowIndex) |>
  summarise(pred_rf = mean(pred), .groups = "drop") |>
  arrange(rowIndex)

svr_oof <- svr_cv$pred |>
  group_by(rowIndex) |>
  summarise(pred_svr = mean(pred), .groups = "drop") |>
  arrange(rowIndex)

xgb_oof <- tibble(
  rowIndex = seq_len(nrow(ml_data)),
  pred_xgb = as.vector(xgb_cv$cv_predict$pred)
)

africa_health <- africa_health |>
  mutate(
    pred_rf  = rf_oof$pred_rf,
    pred_xgb = xgb_oof$pred_xgb,
    pred_svr = svr_oof$pred_svr
  )

# Observed vs Predicted diagnostic plot via mlspatial
plot_obs_vs_pred(
```

```
  observed = africa_health$incidence_rate,
  predicted = africa_health$pred_rf,
  title = "Random Forest CV: Observed vs Predicted"
)
```

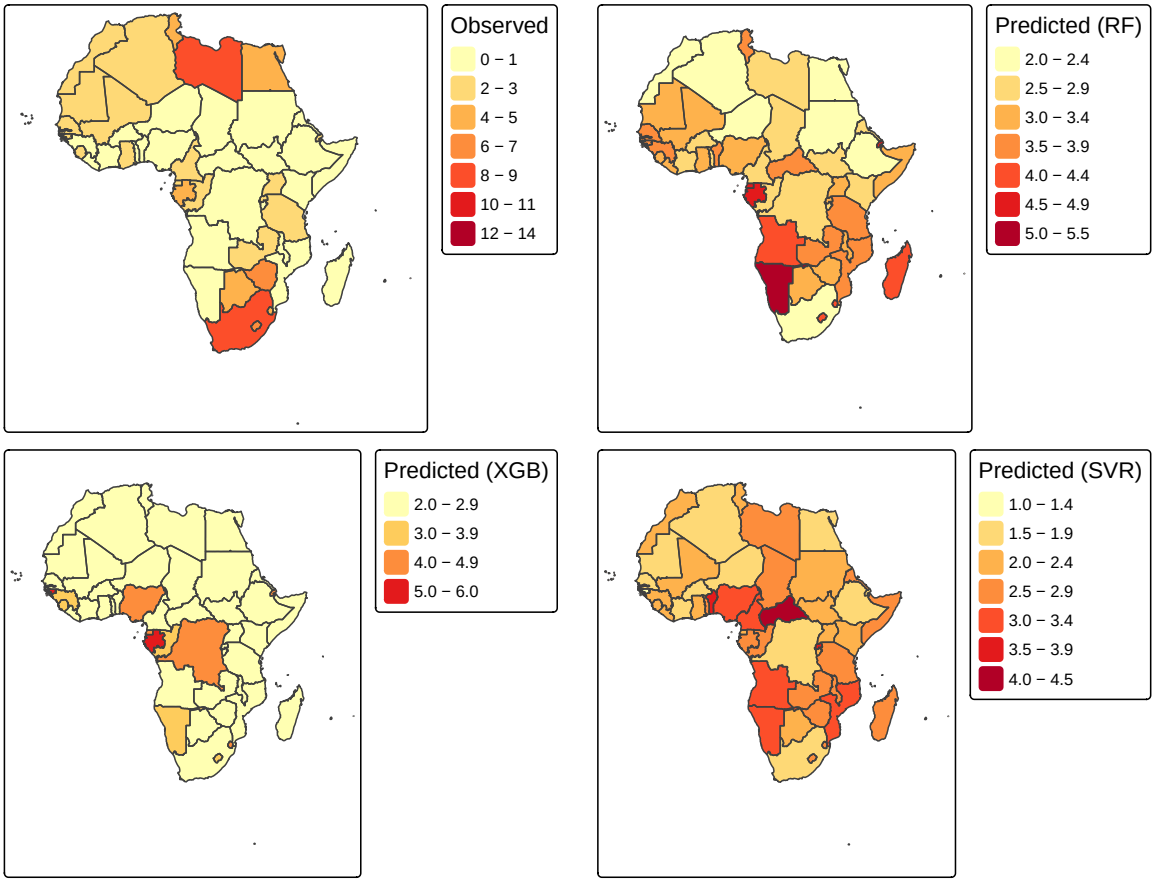



```
plot_obs_vs_pred(
  observed = africa_health$incidence_rate,
  predicted = africa_health$pred_svr,
  title = "SVR CV: Observed vs Predicted"
)
```




```
# Map predictions vs observed
obs_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "incidence_rate",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Observed")
  ) +
  tm_borders()

pred_rf_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "pred_rf",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Predicted (RF)")
  ) +
  tm_borders()
```

```r
pred_xgb_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "pred_xgb",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Predicted (XGB)")
  ) +
  tm_borders()

pred_svr_map <- tm_shape(africa_health) +
  tm_fill(
    fill = "pred_svr",
    fill.scale = tm_scale(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Predicted (SVR)")
  ) +
  tm_borders()

tmap_arrange(obs_map, pred_rf_map, pred_xgb_map, pred_svr_map, ncol = 2)
```



Expected output: - A scatterplot with a 1:1 line and Pearson's r. - Side-by-side maps showing how well the RF, XGB and SVR model reproduces spatial patterns.

Across every method in this module - Random Forest, XGBoost, SVR, the consistent finding is that our six simulated lifestyle/genetic covariates carry no real signal. This is the intended result of a **teaching dataset**: it demonstrates that the entire ML pipeline (data prep → training → tuning → evaluation → spatial pattern mapping) runs correctly and produces internally consistent honestly interpreted null results. When you apply this exact pipeline to real covariate data in your own research, this is the standard of scrutiny that should accompany any reported results.

Module 5 Checklist

☐ Prepared clean ML data with `st_drop_geometry()`

- ☐ Trained Random Forest, XGBoost, and SVR models
- ☐ Evaluated all three models with `eval_model()`
- ☐ Compared RMSE, MAE, and R^2 across algorithms
- ☐ Mapped predictions and identified top risk factors

Next: Proceed to Module 6: Integrated Application from Data to Decision.

6 Module 6: Integrated Application from Data to Decision

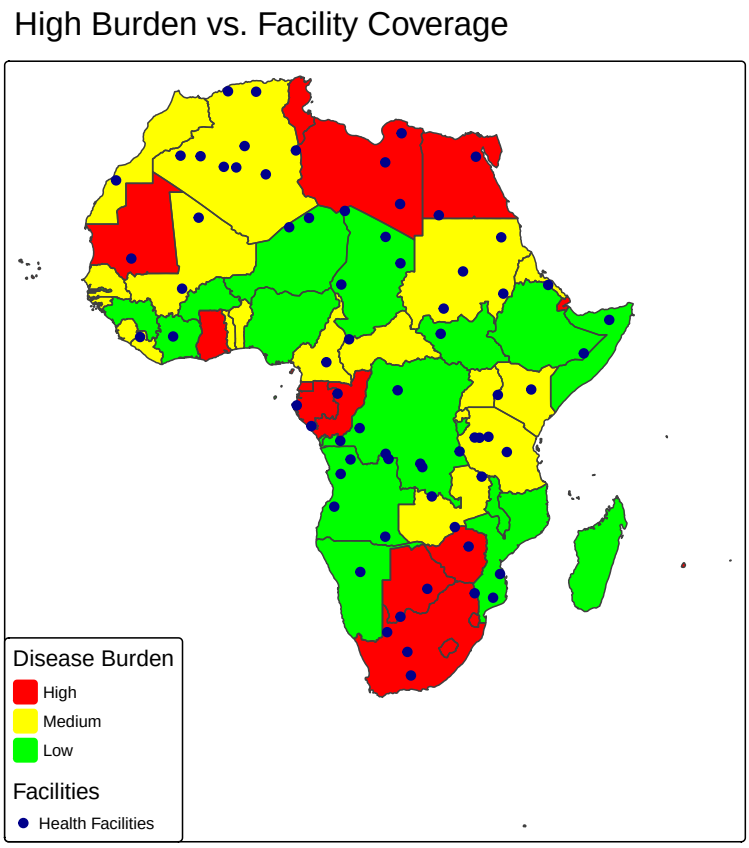
This final module brings everything together. You will overlay disease burden with health facility locations, calculate accessibility distances, generate reproducible reports, and reflect on the ethical dimensions of spatial epidemiology.

6.1 Overlay Disease Burden with Service Availability

Purpose: Identify under-served areas by superimposing disease burden on health facility locations.

```
# Classify burden into tertiles (3-groups) for clearer visual communication
africa_health <- africa_health |>
  mutate(
    burden_class = case_when(
      incidence_rate >= quantile(incidence_rate, 0.67, na.rm = TRUE) ~ "High",
      incidence_rate >= quantile(incidence_rate, 0.33, na.rm = TRUE) ~ "Medium",
      TRUE ~ "Low"
    ),
    burden_class = factor(burden_class, levels = c("High", "Medium", "Low"))
  )

# Simulate 70 health facility points across Africa for demonstration
set.seed(42)
facility_points <- st_sample(africa_health, size = 70, type = "random") |>
  st_as_sf() |>
  mutate(facility_id = paste0("FAC_", row_number()),
         type = "Health Facilities")

decision_map <- tm_shape(africa_health) +
  tm_fill(
```

```
    "burden_class",
    fill.scale = tm_scale(values = c("High"="red","Medium"="yellow","Low"= "green")),
    fill.legend = tm_legend(title = "Disease Burden")
  ) +
  tm_borders() +
  tm_shape(facility_points) +
  tm_dots(
    fill = "type",
    fill.scale = tm_scale_categorical(values ="darkblue"),
    fill.legend = tm_legend(title = "Facilities"),
    size = 0.4
  ) +
  tm_legend(position = c("LEFT", "BOTTOM")) +
  tm_title( "High Burden vs. Facility Coverage")

decision_map
```



Expected output: A map where red (high burden) polygons without nearby blue dots immediately signal priority areas for new facilities.

6.2 Distance / Accessibility Analysis

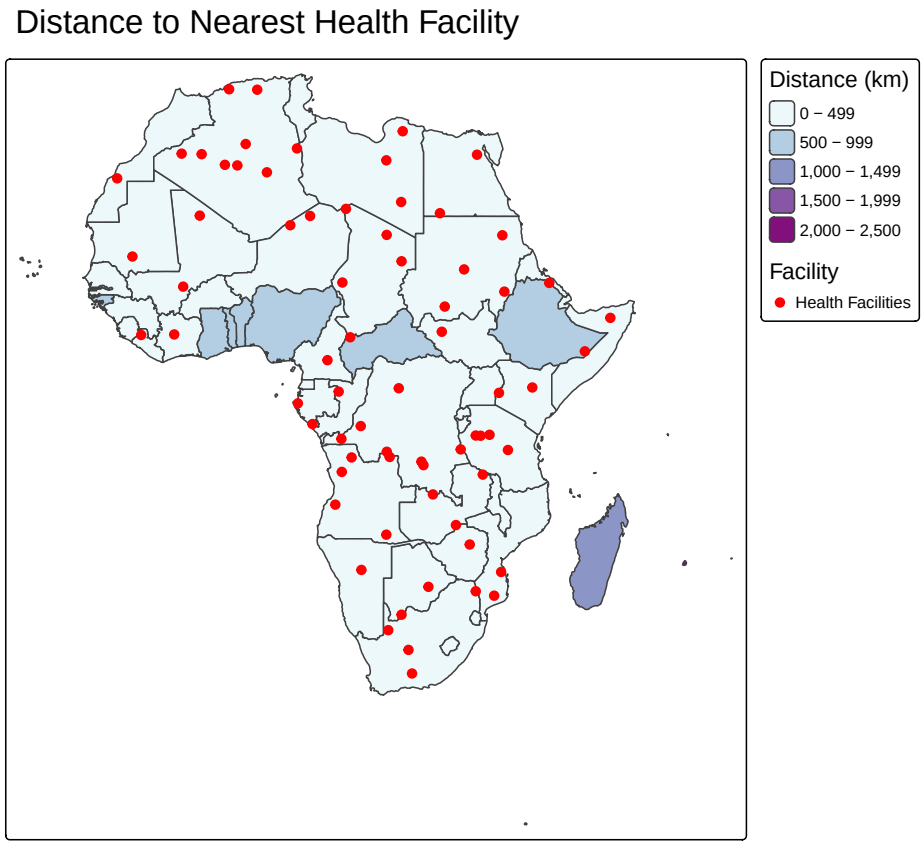
Purpose: Quantify the distance from each administrative unit to the nearest health facility.

```
# Transform both facility point and africa health sfobjects to WGS 84 (EPSG:4326)
# This gives true geodesic distances with st_distance
africa_health_geo <- st_transform(africa_health, crs = 4326)
facility_points_geo <- st_transform(facility_points, crs = 4326)

# Calculate centroid of each polygon (using transformed data)
africa_centroids <- st_centroid(africa_health_geo, of_largest_polygon = TRUE)

# Matrix of geodesic distances (meters) between centroids and facility points
dist_matrix <- st_distance(africa_centroids, facility_points_geo)

# Minimum distance per country (convert to km) and append to original dataset
africa_health <- africa_health |>
  mutate(
    dist_to_facility_km = as.numeric(apply(dist_matrix, 1, min)) / 1000
  )

# Summary statistics
summary(africa_health$dist_to_facility_km)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  28.92  235.75  346.16  458.07  497.54 2380.45
```

```
# Map accessibility
access_map <- tm_shape(africa_health) +
  tm_polygons(
    fill = "dist_to_facility_km",
    fill.scale = tm_scale(values = "brewer.bu_pu"),
    fill.legend = tm_legend(title = "Distance (km)")
        ) +
  tm_shape(facility_points) +
```

```
  tm_dots(
    fill = "type",
    fill.scale = tm_scale_categorical(values ="red"),
    fill.legend = tm_legend(title = "Facility"),
    size = 0.4
  ) +
  tm_title("Distance to Nearest Health Facility")

access_map
```



CRS and Geodesic Distances

When calculating distances, it is crucial to know your CRS. Because our data was transformed to EPSG: 4326 (WGS 84), the sf package uses spherical maths to calculate accurate, real-world geodesic distances, returning the results in meters. Dividing by 1000 gives us our kilometres.

> **Note**
>
> While the spherical engine handles distance perfectly, if you are doing highly localized spatial modelling or calculating exact polygon areas in Ghana, it is still best practice to reproject your data to a metric coordinate system like UTM Zone 30N (EPSG: 32630)

Expected output: - Summary statistics of distance. - A map showing increasing distance as darker purple fills.

6.3 Result Interpretation and Report Generation

Purpose: Produce a reproducible, shareable document integrating code, results, and narrative.

R Markdown (`.Rmd`) and Quarto (`.qmd`) are the gold standards for reproducible research. Below is a minimal Quarto document skeleton you can adapt:

````
---
title: "Spatial Epidemiology Report: Pancreatic Cancer in Africa"
format: html
editor: visual
---

## Introduction
This report analyzes the spatial distribution ...

## Data
```r
#| echo: false
library(mlspatial)
data(africa_shp)
data(panc_incidence)
```

## Results
The Global Moran's I was `r round(global_moran$estimate[1], 3` ...

## Conclusion
High-burden hotspots are concentrated in ...
````

Key features: - **Code chunks** (```` ```{r} ````) execute during rendering. - **Inline code** (‘r code’) updates text automatically when data changes. - **Parametrized reports**: Use YAML `params:` to generate country-specific reports from one template.

Expected output: An HTML or PDF file containing embedded maps, tables, and narrative—fully reproducible by any colleague who has the `.qmd` file and data.

6.4 Ethical Considerations

Purpose: Reflect on methodological biases and data privacy before disseminating findings.

6.4.1 Privacy Protection

- **Aggregated data**: The `mlspatial` built-in data are already aggregated to country level, protecting individual identity. Never map exact household locations or small-area counts (< 5 cases) without differential privacy or aggregation.
- **Re-identification risk**: Combining spatial coordinates with demographic details (age, sex, date of diagnosis) can re-identify patients in small villages.

6.4.2 Modifiable Areal Unit Problem (MAUP)

- **Definition**: Results depend on the boundaries chosen. Country-level analysis may hide sub-national hotspots; conversely, fine-scale analysis may produce unstable rates due to small populations.
- **Mitigation**: Conduct sensitivity analyses at multiple scales (district, region, country) and report how conclusions change.

6.4.3 Ecological Fallacy

- **Definition**: Relationships observed at the aggregate level do not necessarily hold for individuals. For example, a country with high average temperature and high cancer incidence does not prove that heat causes cancer in individual patients.
- **Mitigation**: Use language like “areas with higher temperature are associated with higher incidence” rather than “temperature causes cancer.”

6.4.4 Spatial Inequality & Resource Allocation

- Maps are political. Highlighting "high burden" regions can attract funding, but it can also stigmatize communities. Always:
    1. Share results with local health officials before publication.
    2. Provide confidence intervals or classification uncertainty.
    3. Pair burden maps with resource maps (as in Section 6.1) to focus on actionable gaps.

Closing Remarks

You have now completed a full spatial epidemiology workflow in **R only**:

1. Imported and joined spatial and health data
2. Created static, interactive, and multi-layer thematic maps
3. Detected global and local spatial autocorrelation
4. Built OLS and Geographically Weighted Regression models
5. Trained and evaluated Random Forest, XGBoost, and SVR models via `mlspatial`
6. Integrated disease burden with service availability and generated reproducible reports

Next steps for the Ghana R User Community: - Replace the built-in Africa data with Ghana Health Service district-level data. - Download real WorldClim rasters via `geodata::worldclim_global()`. - Explore `mlspatial` vignettes: `vignette("mlspatial")` after installing the package.

Reference: Azeez, A., & Noel, C. (2025). *Predictive Modelling and Spatial Distribution of Pancreatic Cancer in Africa Using Machine Learning-Based Spatial Model*. Zenodo. https://doi.org/10.5281/zenodo.16529986